

MANUAL COMPLETO DE MATEMÁTICAS I

Grado en Ingeniería Informática - 1º Curso

Este documento unifica todos los temas del plan de estudio en un único manual para facilitar su lectura, impresión o conversión a formatos como PDF.

Índice General

- **Bloque 1: Herramientas Básicas y Sucesiones**
 - [Tema 1: Números Complejos](#)
 - [Tema 2: Sucesiones Numéricas](#)
 - [Tema 3: Series Matemáticas](#)
- **Bloque 2: Cálculo Diferencial en una Variable**
 - [Tema 4: Cálculo de Límites y Continuidad](#)
 - [Tema 5: Derivación y Aplicaciones](#)
 - [Tema 6: Polinomio de Taylor](#)
- **Bloque 3: Cálculo Integral**
 - [Tema 7: Integración](#)
- **Bloque 4: Cálculo Numérico (El toque informático)**
 - [Tema 8: Resolución Numérica de Ecuaciones](#)
 - [Tema 9: Interpolación Polinómica](#)
 - [Tema 10: Integración Numérica](#)
- **Secciones Finales**
 - [Glosario de Términos](#)
 - [Bibliografía Recomendada](#)

Tema 1: Números Complejos

Los números complejos amplían el conjunto de los números reales para permitir soluciones a ecuaciones que carecen de ellas en el dominio real, como $x^2 + 1 = 0$. En la ingeniería informática, su aplicación se extiende desde la renderización de gráficos 2D y el procesamiento de señales digitales (transformadas de Fourier) hasta la computación cuántica y la simulación física.

1. Definición y Representación Geométrica en $\mathbb{C}$

Definimos el conjunto de los **números complejos** (denotado por $\mathbb{C}$) como el conjunto de todos los pares ordenados (a, b) de números reales, dotado de dos operaciones fundamentales: la suma y la multiplicación.

Introducimos la **unidad imaginaria**, denotada por i , que satisface la propiedad fundamental:

$$i^2 = -1 \quad \Rightarrow \quad i = \sqrt{-1}$$

Un número complejo $z \in \mathbb{C}$ se representa comúnmente en su **forma binómica**:

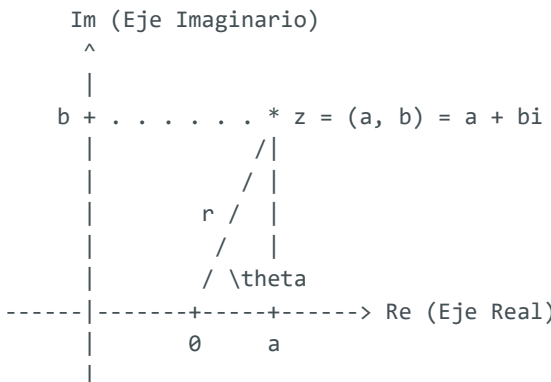
$$z = a + bi$$

donde:

- $\text{Re}(z) = a$ es la **parte real** de z .
- $\text{Im}(z) = b$ es la **parte imaginaria** de z .

Plano Complejo (Plano de Argand)

Geométricamente, representamos $\mathbb{C}$ como un plano bidimensional, donde el eje horizontal representa la parte real (eje real) y el eje vertical representa la parte imaginaria (eje imaginario). Cada número complejo $z = a + bi$ se asocia con un punto de coordenadas (a, b) , conocido como **afijo** de z .



2. Formas de Representación

Un número complejo $z = a + bi$ puede expresarse en tres formas matemáticas fundamentales que facilitan diferentes tipos de cálculos algebraicos y geométricos.

2.1 Forma Binómica

$$z = a + bi$$

Es idónea para operaciones de suma y resta.

2.2 Forma Polar (o Trigonométrica)

Definimos el **módulo** $r = |z|$ como la distancia desde el origen al afijo (a, b) :

$$r = |z| = \sqrt{a^2 + b^2}$$

Definimos el **argumento** $\theta = \arg(z)$ como el ángulo medido en radianes desde el eje real positivo en sentido antihorario:

$$\theta = \arctan\left(\frac{b}{a}\right)$$

[!WARNING] La función $\arctan(b/a)$ ordinaria devuelve valores en el rango $[-\pi/2, \pi/2]$. Debemos ajustar el cuadrante del ángulo según los signos de a y b :

- **I Cuadrante** ($a > 0, b \geq 0$): $\theta = \arctan(b/a)$
- **II Cuadrante** ($a < 0, b \geq 0$): $\theta = \arctan(b/a) + \pi$
- **III Cuadrante** ($a < 0, b < 0$): $\theta = \arctan(b/a) - \pi$ (o $\arctan(b/a) + \pi$)
- **IV Cuadrante** ($a > 0, b < 0$): $\theta = \arctan(b/a)$

Con r y θ , la forma trigonométrica es:

$$z = r(\cos \theta + i \sin \theta)$$

Se abrevia como $z = r_{\theta}$.

2.3 Forma Exponencial

Basada en la **fórmula de Euler**:

$$e^{i\theta} = \cos \theta + i \sin \theta$$

Cualquier número complejo z se escribe de forma exponencial como:

$$z = re^{i\theta}$$

Esta forma es extremadamente potente para multiplicaciones, divisiones y potencias, ya que hereda las propiedades normales de los exponentes exponenciales.

3. Operaciones Aritméticas en $\mathbb{C}$

Sean $z_1 = a + bi = r_1 e^{i\theta_1}$ y $z_2 = c + di = r_2 e^{i\theta_2}$.

3.1 Suma y Resta (Forma Binómica)

Se suman o restan las partes reales y las partes imaginarias por separado:

$$z_1 \pm z_2 = (a \pm c) + (b \pm d)i$$

3.2 Multiplicación

- En forma binómica:

$$z_1 \cdot z_2 = (a + bi)(c + di) = ac + adi + bci + bdi^2 = (ac - bd) + (ad + bc)i$$

- En forma polar/exponencial:

$$z_1 \cdot z_2 = (r_1 e^{i\theta_1})(r_2 e^{i\theta_2}) = (r_1 r_2) e^{i(\theta_1 + \theta_2)}$$

(Multiplicamos los módulos y sumamos los argumentos).

3.3 División

- En forma binómica (multiplicando por el conjugado del denominador): El conjugado de $z_2 = c + di$ es $\bar{z}_2 = c - di$.

$$\frac{z_1}{z_2} = \frac{a + bi}{c + di} \cdot \frac{c - di}{c - di} = \frac{(ac + bd) + (bc - ad)i}{c^2 + d^2} = \frac{ac + bd}{c^2 + d^2} + \frac{bc - ad}{c^2 + d^2}i$$

- En forma polar/exponencial:

$$\frac{z_1}{z_2} = \frac{r_1 e^{i\theta_1}}{r_2 e^{i\theta_2}} = \left(\frac{r_1}{r_2}\right) e^{i(\theta_1 - \theta_2)}$$

(Dividimos los módulos y restamos los argumentos).

3.4 Potenciación (Fórmula de De Moivre)

Para un número entero n :

$$z^n = (r(\cos \theta + i \sin \theta))^n = r^n (\cos(n\theta) + i \sin(n\theta))$$

O en forma exponencial:

$$z^n = (re^{i\theta})^n = r^n e^{in\theta}$$

4. Radicación en C

Dado $z = re^{i\theta}$, buscamos las soluciones $w = Re^{i\phi}$ a la ecuación $w^n = z$. Por la fórmula de De Moivre:

$$R^n e^{in\phi} = r e^{i\theta}$$

Esto implica que:

- $R^n = r \Rightarrow R = \sqrt[n]{r}$ (raíz real positiva del módulo).
- $n\phi = \theta + 2k\pi \Rightarrow \phi_k = \frac{\theta + 2k\pi}{n}$ para $k = 0, 1, 2, \dots, n-1$.

Por tanto, un número complejo $z \neq 0$ tiene exactamente n **raíces n -ésimas** distintas, denotadas por w_k :

$$w_k = \sqrt[n]{r} \exp\left(i \frac{\theta + 2k\pi}{n}\right) \quad \text{para } k = 0, 1, \dots, n-1$$

Interpretación Geométrica

Los afijos de las n raíces de un número complejo se sitúan sobre una circunferencia de radio $\sqrt[n]{r}$ centrada en el origen y forman los vértices de un **polígono regular de n lados**.

5. El Toque Informático

5.1 Rotaciones en Gráficos Computacionales 2D

En gráficos bidimensionales, rotar un punto (x, y) un ángulo α alrededor del origen es algebraicamente idéntico a multiplicar el número complejo $p = x + yi$ por el número complejo unitario de rotación $R = e^{i\alpha} = \cos \alpha + i \sin \alpha$:

$$p' = p \cdot R = (x + yi)(\cos \alpha + i \sin \alpha) = (x \cos \alpha - y \sin \alpha) + (x \sin \alpha + y \cos \alpha)i$$

Esto produce exactamente las coordenadas de la matriz de rotación 2D estándar:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

5.2 Generación de Fractales (El Conjunto de Julia)

El conjunto de Julia se define mediante la iteración de la función compleja cuadratura:

$$z_{n+1} = z_n^2 + c$$

donde $c \in \mathbb{C}$ es una constante. Dependiendo del valor inicial z_0 , la sucesión puede divergir al infinito o permanecer acotada.

A continuación, implementamos en Python una visualización del conjunto de Julia para mostrar el poder del plano complejo en la generación de patrones auto-similares.

```
import numpy as np
import matplotlib.pyplot as plt

def generar_julia(c, ancho=800, alto=800, iter_max=300):
    # Definimos el plano complejo
    x = np.linspace(-1.5, 1.5, ancho)
    y = np.linspace(-1.5, 1.5, alto)
    x_grid, y_grid = np.meshgrid(x, y)

    # Creamos la matriz de números complejos  $z = x + iy$ 
    z = x_grid + 1j * y_grid

    # Matriz para almacenar en qué iteración diverge cada punto
    fractal = np.zeros(z.shape, dtype=int)

    # Máscara lógica para los puntos que no han divergido
    activo = np.ones(z.shape, dtype=bool)

    for i in range(iter_max):
        # Iteración  $z = z^2 + c$  solo para puntos activos
        z[activo] = z[activo]**2 + c

        # Un punto diverge si su módulo supera 2 (módulo al cuadrado > 4)
        diverge = np.abs(z) > 2.0

        # Guardamos la iteración de divergencia
        fractal[diverge & activo] = i

        # Desactivamos los puntos divergentes
        activo[diverge] = False

    return fractal

# Constante de Julia clásica (puedes probar con otros valores como -0.8 + 0.156j)
c_julia = -0.7 + 0.27015j
fractal = generar_julia(c_julia)

# Visualización
plt.figure(figsize=(8, 8))
plt.imshow(fractal, cmap='twilight_shifted', extent=[-1.5, 1.5, -1.5, 1.5])
plt.colorbar(label='Velocidad de divergencia (iteraciones)')
plt.title(f"Conjunto de Julia para  $c = {c_julia}$ ")
plt.xlabel("Re(z)")
plt.ylabel("Im(z)")
plt.show()
```

6. Ejercicios Resueltos

Ejercicio 1

Calcular todas las raíces cúbicas de $z = -8i$ y representarlas geoméricamente.

Solución:

1. Expresar z en forma polar:

- Módulo: $r = |-8i| = \sqrt{0^2 + (-8)^2} = 8$.
- Argumento: El número está sobre el eje imaginario negativo, por tanto, $\theta = \frac{3\pi}{2}$ (o $-\frac{\pi}{2}$).
- Entonces, $z = 8e^{i\frac{3\pi}{2}}$.

2. Calcular las raíces cúbicas usando la fórmula de radicación:

$$w_k = \sqrt[3]{8} \exp\left(i \frac{\frac{3\pi}{2} + 2k\pi}{3}\right) \quad \text{para } k = 0, 1, 2$$

$$w_k = 2 \exp\left(i \left(\frac{\pi}{2} + \frac{2k\pi}{3}\right)\right)$$

3. Evaluar para cada valor de k :

- $k = 0$:

$$w_0 = 2e^{i\pi/2} = 2(\cos(\pi/2) + i \sin(\pi/2)) = 2(0 + i) = 2i$$

- $k = 1$:

$$w_1 = 2e^{i(\pi/2 + 2\pi/3)} = 2e^{i(7\pi/6)}$$

$$w_1 = 2\left(\cos \frac{7\pi}{6} + i \sin \frac{7\pi}{6}\right) = 2\left(-\frac{\sqrt{3}}{2} - \frac{1}{2}i\right) = -\sqrt{3} - i$$

- $k = 2$:

$$w_2 = 2e^{i(\pi/2 + 4\pi/3)} = 2e^{i(11\pi/6)}$$

$$w_2 = 2\left(\cos \frac{11\pi}{6} + i \sin \frac{11\pi}{6}\right) = 2\left(\frac{\sqrt{3}}{2} - \frac{1}{2}i\right) = \sqrt{3} - i$$

4. Representación geométrica: Los afijos $w_0 = 2i$, $w_1 = -\sqrt{3} - i$, $w_2 = \sqrt{3} - i$ están en un círculo de radio 2 y forman los vértices de un triángulo equilátero.

7. Ejercicios Propuestos

1. Expresar el número complejo $z = -1 + \sqrt{3}i$ en forma polar y exponencial, y hallar z^6 usando la fórmula de De Moivre.
2. Resolver en $\mathbb{C}$ la ecuación polinómica $z^4 + 16 = 0$.

3. Demostrar que para cualquier $z \in \mathbb{C}$, el producto $z \cdot \bar{z}$ es un número real no negativo igual a $|z|^2$.

Tema 2: Sucesiones Numéricas

Una sucesión es una función cuyo dominio es el conjunto de los números naturales ($\mathbb{N}$) y cuyo codominio es el de los números reales ($\mathbb{C}$ o $\mathbb{R}$). En informática, las sucesiones describen el comportamiento paso a paso de los algoritmos iterativos, la asignación dinámica de memoria y la complejidad temporal asintótica de los programas.

1. Concepto de Límite y Convergencia

Una **sucesión real** es una lista ordenada de números reales denotada por $\{a_n\}_{n=1}^{\infty}$ o simplemente por (a_n) :

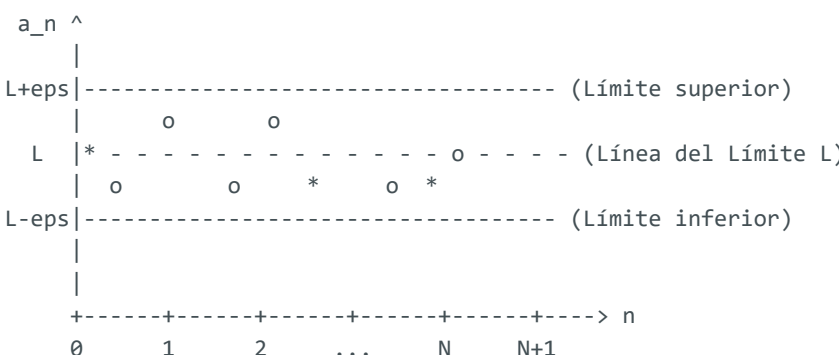
$$a_1, a_2, a_3, \dots, a_n, \dots$$

donde a_n representa el término general o n -ésimo término de la sucesión.

Definición Formal de Límite

Se dice que la sucesión $\{a_n\}$ converge a un límite $L \in \mathbb{R}$ si, para cualquier valor de tolerancia $\varepsilon > 0$, existe un número natural $N \in \mathbb{N}$ tal que para todo término posterior a N ($n > N$), la distancia entre el término de la sucesión y el límite es menor que ε :

$$\lim_{n \rightarrow \infty} a_n = L \Leftrightarrow \forall \varepsilon > 0, \exists N \in \mathbb{N} \text{ tal que } \forall n > N, |a_n - L| < \varepsilon$$



- **Sucesiones Convergentes:** Tienen un límite finito $L \in \mathbb{R}$.
- **Sucesiones Divergentes:** Su límite es $+\infty$ o $-\infty$.
- **Sucesiones Oscilantes:** No tienden a ningún valor fijo ni divergen al infinito (por ejemplo, $a_n = (-1)^n$).

2. Cálculo de Límites y Resolución de Indeterminaciones

Al operar con límites de sucesiones podemos encontrarnos con expresiones cuyo resultado no es evidente directamente. Estas se conocen como **indeterminaciones** y las principales son:

$$\frac{\infty}{\infty}, \quad \frac{0}{0}, \quad \infty - \infty, \quad 0 \cdot \infty, \quad 1^\infty, \quad \infty^0, \quad 0^0$$

2.1 Indeterminación $\frac{\infty}{\infty}$ en Polinomios

Si $a_n = P(n)$ y $b_n = Q(n)$ son polinomios de grados p y q , entonces:

$$\lim_{n \rightarrow \infty} \frac{P(n)}{Q(n)} = \begin{cases} \pm\infty & \text{si } p > q \\ \frac{\text{coeficiente principal de } P}{\text{coeficiente principal de } Q} & \text{si } p = q \\ 0 & \text{si } p < q \end{cases}$$

2.2 Indeterminación 1^∞ (Número e)

Si una sucesión tiene la forma $(u_n)^{v_n}$ donde $\lim_{n \rightarrow \infty} u_n = 1$ y $\lim_{n \rightarrow \infty} v_n = \infty$, se utiliza la equivalencia asociada al número e :

$$\lim_{n \rightarrow \infty} (u_n)^{v_n} = e^{\lim_{n \rightarrow \infty} v_n (u_n - 1)}$$

3. Criterios Avanzados de Convergencia

Cuando el término general de la sucesión no es simple, se recurre a criterios específicos.

3.1 Criterio de la Cota de la Razón (o del Sándwich / Compresión)

Sean $\{a_n\}$, $\{b_n\}$ y $\{c_n\}$ sucesiones tales que $a_n \leq b_n \leq c_n$ para todo $n \geq N_0$. Si $\lim_{n \rightarrow \infty} a_n = \lim_{n \rightarrow \infty} c_n = L$, entonces:

$$\lim_{n \rightarrow \infty} b_n = L$$

3.2 Criterio de Stolz-Cesàro (La Regla de L'Hôpital para Sucesiones)

Sea $\{x_n\}$ una sucesión cualquiera y $\{y_n\}$ una sucesión estrictamente creciente y divergente a $+\infty$ ($\lim_{n \rightarrow \infty} y_n = +\infty$). Si existe el límite:

$$\lim_{n \rightarrow \infty} \frac{x_n - x_{n-1}}{y_n - y_{n-1}} = L \quad (L \in \mathbb{R} \cup \{\pm\infty\})$$

Entonces:

$$\lim_{n \rightarrow \infty} \frac{x_n}{y_n} = L$$

3.3 Criterio de la Media Aritmética

Si $\lim_{n \rightarrow \infty} a_n = L$, entonces la sucesión de sus medias aritméticas también converge a L :

$$\lim_{n \rightarrow \infty} \frac{a_1 + a_2 + \dots + a_n}{n} = L$$

3.4 Criterio de la Media Geométrica (para términos positivos)

Si $\lim_{n \rightarrow \infty} a_n = L$ (con $a_n > 0$), entonces la sucesión de sus medias geométricas también converge a L :

$$\lim_{n \rightarrow \infty} \sqrt[n]{a_1 \cdot a_2 \cdot \dots \cdot a_n} = L$$

4. El Toque Informático

4.1 Complejidad Temporal Asintótica (Notación O Grande)

En ciencias de la computación, la complejidad temporal de un algoritmo se mide con la **Notación Big-O** (O), la cual es conceptualmente el límite de una sucesión. Decimos que el número de operaciones de un programa con entrada de tamaño n es de orden $g(n)$, denotado por $f(n) = O(g(n))$, si existe una constante $C > 0$ y un umbral N_0 tal que para toda entrada mayor que el umbral, el coste no supera a $C \cdot g(n)$:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = K < \infty$$

Por ejemplo, comparar un algoritmo de ordenamiento de burbuja ($O(n^2)$) con MergeSort ($O(n \log n)$) equivale a analizar el comportamiento asintótico cuando $n \rightarrow \infty$:

$$\lim_{n \rightarrow \infty} \frac{n \log n}{n^2} = \lim_{n \rightarrow \infty} \frac{\log n}{n} = 0$$

Esto demuestra formalmente que, para entradas grandes, MergeSort es infinitamente más eficiente que el ordenamiento de burbuja.

4.2 Análisis de Recurrencias e Iteración: La Sucesión de Fibonacci

Muchas sucesiones en informática se definen de forma recursiva (ecuaciones en diferencias). La más emblemática es la sucesión de Fibonacci:

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad \text{para } n \geq 2$$

Analíticamente, el término general viene dado por la **Fórmula de Binet**:

$$F_n = \frac{\phi^n - \psi^n}{\sqrt{5}}$$

donde $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ (el número áureo) y $\psi = \frac{1-\sqrt{5}}{2} \approx -0.618$. Esto implica que el límite del cociente de dos términos consecutivos es precisamente la razón áurea:

$$\lim_{n \rightarrow \infty} \frac{F_{n+1}}{F_n} = \phi$$

A continuación, implementamos en Python tres enfoques de cálculo de la sucesión de Fibonacci para analizar cómo la elección algorítmica afecta a la complejidad de la sucesión de tiempos de ejecución.

```
import time
import numpy as np
import matplotlib.pyplot as plt

# 1. Enfoque Recursivo Ineficiente: O(phi^n)
def fib_recursivo(n):
    if n <= 1:
        return n
    return fib_recursivo(n-1) + fib_recursivo(n-2)

# 2. Enfoque Iterativo con Programación Dinámica: O(n)
def fib_iterativo(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

# 3. Enfoque de Exponenciación Matricial: O(log n)
# Basado en la relación [[F_{n+1}, F_n], [F_n, F_{n-1}]] = [[1, 1], [1, 0]]^n
def fib_matricial(n):
    if n == 0: return 0
    F = np.array([[1, 1], [1, 0]], dtype=object)
    def power(A, p):
        res = np.eye(2, dtype=object)
        base = A
        while p > 0:
            if p % 2 == 1:
                res = np.dot(res, base)
            base = np.dot(base, base)
            p //= 2
    return fib_matricial(n-1) + fib_matricial(n-2)
```

```

        return res
    return power(F, n - 1)[0, 0]

# Comparación de tiempos de ejecución
rango_n = list(range(10, 35))
tiempos_rec = []
tiempos_it = []

for n in rango_n:
    # Tiempo recursivo
    t0 = time.perf_counter()
    fib_recursivo(n)
    tiempos_rec.append(time.perf_counter() - t0)

    # Tiempo iterativo
    t0 = time.perf_counter()
    fib_iterativo(n)
    tiempos_it.append(time.perf_counter() - t0)

# Gráfico de tiempos
plt.figure(figsize=(9, 5))
plt.plot(rango_n, tiempos_rec, label="Recursivo: O(1.618^n)", marker='o', color='red')
plt.plot(rango_n, tiempos_it, label="Iterativo: O(n)", marker='s', color='blue')
plt.yscale('log')
plt.xlabel("Término n de Fibonacci")
plt.ylabel("Tiempo de ejecución (segundos, escala logarítmica)")
plt.title("Estudio de Complejidad: Algoritmos para Fibonacci")
plt.legend()
plt.grid(True, which="both", ls="--")
plt.show()

```

5. Ejercicios Resueltos

Ejercicio 1

Calcular el límite de la sucesión:

$$\lim_{n \rightarrow \infty} \frac{1^2 + 2^2 + \dots + n^2}{n^3}$$

Solución utilizando el Criterio de Stolz-Cesàro: Identificamos $x_n = 1^2 + 2^2 + \dots + n^2$ e $y_n = n^3$.

Dado que $\{y_n\}$ es estrictamente creciente y tiende a $+\infty$, aplicamos el criterio:

$$\lim_{n \rightarrow \infty} \frac{x_n - x_{n-1}}{y_n - y_{n-1}} = \lim_{n \rightarrow \infty} \frac{(1^2 + \dots + n^2) - (1^2 + \dots + (n-1)^2)}{n^3 - (n-1)^3}$$

El numerador se simplifica directamente al término n -ésimo:

$$x_n - x_{n-1} = n^2$$

El denominador se expande mediante productos notables:

$$y_n - y_{n-1} = n^3 - (n^3 - 3n^2 + 3n - 1) = 3n^2 - 3n + 1$$

Sustituyendo en el límite:

$$\lim_{n \rightarrow \infty} \frac{n^2}{3n^2 - 3n + 1}$$

Como los polinomios del numerador y denominador tienen el mismo grado (grado 2), el límite es el cociente de sus coeficientes principales:

$$\lim_{n \rightarrow \infty} \frac{n^2}{3n^2 - 3n + 1} = \frac{1}{3}$$

Por tanto, de acuerdo con el criterio de Stolz-Cesàro:

$$\lim_{n \rightarrow \infty} \frac{1^2 + 2^2 + \dots + n^2}{n^3} = \frac{1}{3}$$

6. Ejercicios Propuestos

1. Calcular el límite de la sucesión:

$$\lim_{n \rightarrow \infty} \left(\frac{2n-3}{2n+5} \right)^{4n+1}$$

2. Utilizar el criterio de Stolz-Cesàro para calcular:

$$\lim_{n \rightarrow \infty} \frac{\ln(n!)}{n \ln n}$$

(Pista: Puedes usar la fórmula de Stirling o el límite del cociente de logaritmos).

3. Demostrar la convergencia de la sucesión recursiva definida por $a_1 = \sqrt{2}$ y $a_{n+1} = \sqrt{2 + a_n}$ para $n \geq 1$, y calcular su límite.

Tema 3: Series Matemáticas

Una serie matemática es la suma de los términos de una sucesión infinita. En informática, las series sustentan los algoritmos de aproximación numérica, la teoría de la información y los métodos de discretización de señales. Además, su cálculo práctico introduce un desafío clave en la computación: las limitaciones de precisión de la aritmética de coma flotante.

1. Definición y Concepto de Convergencia

Dada una sucesión $\{a_n\}_{n=1}^{\infty}$, definimos la **serie matemática** asociada como la suma formal de sus infinitos términos:

$$\sum_{n=1}^{\infty} a_n = a_1 + a_2 + a_3 + \dots$$

Para estudiar su validez matemática, definimos la sucesión de **sumas parciales** $\{S_N\}_{N=1}^{\infty}$:

$$S_N = \sum_{n=1}^N a_n = a_1 + a_2 + \dots + a_N$$

- Si la sucesión de sumas parciales $\{S_N\}$ converge a un límite finito $S \in \mathbb{R}$ cuando $N \rightarrow \infty$, decimos que la serie es **convergente** y que su suma es S :

$$\sum_{n=1}^{\infty} a_n = \lim_{N \rightarrow \infty} S_N = S$$

- Si $\lim_{N \rightarrow \infty} S_N = \pm\infty$, la serie es **divergente**.
- Si el límite de $\{S_N\}$ no existe, la serie es **oscilante**.

[!IMPORTANT] Condición Necesaria de Convergencia (Criterio del Término General): Si la serie $\sum_{n=1}^{\infty} a_n$ es convergente, entonces el límite de su término general cuando $n \rightarrow \infty$ debe ser cero:

$$\lim_{n \rightarrow \infty} a_n = 0$$

Atención: El recíproco no es cierto (por ejemplo, la serie armónica cumple que $\lim_{n \rightarrow \infty} 1/n = 0$, pero es divergente). Si el límite no es cero, podemos afirmar inmediatamente que la serie diverge.

2. Series Notables

Existen ciertas series cuyo comportamiento y suma exacta se conocen de antemano y sirven como referencia de comparación.

2.1 Series Geométricas

Tienen la forma:

$$\sum_{n=0}^{\infty} ar^n = a + ar + ar^2 + ar^3 + \dots \quad (\text{donde } a \neq 0)$$

El número r es la **razón** de la serie.

- **Convergencia:** Converge si y solo si $|r| < 1$.
- **Suma:** Si converge, su suma es:

$$S = \frac{a}{1 - r}$$

2.2 Series Armónicas y Armónicas Generalizadas (p -series)

La **serie armónica** es:

$$\sum_{n=1}^{\infty} \frac{1}{n} = 1 + \frac{1}{2} + \frac{1}{3} + \dots \quad (\text{Divergente})$$

La **serie armónica generalizada** (p -serie) tiene la forma:

$$\sum_{n=1}^{\infty} \frac{1}{n^p}$$

- **Convergencia:** Converge si y solo si $p > 1$.
- **Divergencia:** Diverge si $p \leq 1$.

3. Criterios de Convergencia para Series de Términos Positivos ($a_n \geq 0$)

Cuando no es posible calcular de forma directa la suma parcial de la serie, determinamos su convergencia mediante criterios cualitativos.

3.1 Criterio de Comparación Directa

Sean $\sum a_n$ y $\sum b_n$ series de términos positivos tales que $a_n \leq b_n$ para todo $n \geq N_0$:

- Si $\sum b_n$ converge $\Rightarrow \sum a_n$ converge.
- Si $\sum a_n$ diverge $\Rightarrow \sum b_n$ diverge.

3.2 Criterio de Comparación por Límite

Sean $\sum a_n$ y $\sum b_n$ series de términos positivos. Si existe el límite:

$$\lim_{n \rightarrow \infty} \frac{a_n}{b_n} = L \quad (0 < L < \infty)$$

Ambas series tienen el mismo comportamiento (ambas convergen o ambas divergen).

3.3 Criterio del Cociente (o de D'Alembert)

Sea $\sum a_n$ una serie de términos positivos. Supongamos que existe el límite:

$$\lim_{n \rightarrow \infty} \frac{a_{n+1}}{a_n} = L$$

- Si $L < 1$, la serie converge.
- Si $L > 1$ (o $L = \infty$), la serie diverge.
- Si $L = 1$, el criterio no es concluyente (se debe usar otro método).

3.4 Criterio de la Raíz (o de Cauchy)

Sea $\sum a_n$ una serie de términos positivos. Supongamos que existe el límite:

$$\lim_{n \rightarrow \infty} \sqrt[n]{a_n} = L$$

- Si $L < 1$, la serie converge.
- Si $L > 1$, la serie diverge.
- Si $L = 1$, el criterio no es concluyente.

3.5 Criterio de la Integral (de Maclaurin-Cauchy)

Sea $f(x)$ una función continua, positiva y decreciente en el intervalo $[1, \infty)$ tal que $f(n) = a_n$. Entonces:

$$\sum_{n=1}^{\infty} a_n \text{ converge} \Leftrightarrow \int_1^{\infty} f(x) dx \text{ es convergente (finito)}$$

4. Series Alternadas e Integración de Signo

Una **serie alternada** es aquella cuyos términos cambian alternativamente de signo:

$$\sum_{n=1}^{\infty} (-1)^{n-1} a_n = a_1 - a_2 + a_3 - a_4 + \dots \quad (\text{con } a_n > 0)$$

Criterio de Leibniz para Series Alternadas

La serie alternada $\sum_{n=1}^{\infty} (-1)^{n-1} a_n$ converge si cumple las dos condiciones siguientes:

1. La sucesión $\{a_n\}$ es monótona decreciente: $a_{n+1} \leq a_n$ para todo $n \geq N_0$.
2. El límite del término general es cero: $\lim_{n \rightarrow \infty} a_n = 0$.

Convergencia Absoluta y Condicional

- Decimos que una serie $\sum a_n$ **converge absolutamente** si la serie de sus valores absolutos $\sum |a_n|$ es convergente.
- Decimos que $\sum a_n$ **converge condicionalmente** si la serie es convergente, pero la serie de sus valores absolutos $\sum |a_n|$ es divergente (por ejemplo, la serie armónica alternada $\sum \frac{(-1)^{n-1}}{n}$).

5. El Toque Informático

5.1 Precisión de Coma Flotante (IEEE 754) e Inestabilidad en la Suma de Series

En un ordenador, los números reales se representan bajo el estándar **IEEE 754** usando mantisa y exponente. La limitación del número de bits de la mantisa introduce el **error de redondeo**. Al sumar una serie infinita numéricamente (por ejemplo, sumando términos en un bucle), si sumamos un término muy pequeño a una suma acumulada grande, el exponente del término pequeño se ajusta para alinearse con el de la suma. Al desplazar la mantisa a la derecha, los bits menos significativos del término pequeño se pierden por completo. Este fenómeno se conoce como **pérdida de significación por absorción**.

5.2 El Algoritmo de Suma de Kahan

Para mitigar la acumulación de errores de redondeo en bucles que suman grandes series de números, William Kahan diseñó la **Suma de Kahan**. Este algoritmo almacena una variable correctora para "recordar" el error perdido en el paso anterior y añadirlo compensado en la siguiente iteración.

A continuación, implementamos en Python una simulación que calcula la suma de la famosa **Serie de Basel** (cuyo valor exacto es $\pi^2/6 \approx 1.6449340668482264$):

$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$$

Comparamos una suma ordinaria en coma flotante simple de 32 bits (para amplificar y hacer notable la pérdida de precisión en menos términos) frente a la Suma de Kahan.

```
import numpy as np

# Definimos el número de términos para aproximar
N = 10_000_000

# Valor teórico exacto de la serie de Basel
valor_teorico = (np.pi ** 2) / 6

# 1. Suma ordinaria de precisión simple (float32)
def suma_ordinaria(n_terminos):
    suma = np.float32(0.0)
    for i in range(1, n_terminos + 1):
        termino = np.float32(1.0 / (i * i))
        suma += termino
    return suma

# 2. Algoritmo de Suma de Kahan (float32)
def suma_kahan(n_terminos):
    suma = np.float32(0.0)
    c = np.float32(0.0) # Variable para compensar el error perdido
    for i in range(1, n_terminos + 1):
        termino = np.float32(1.0 / (i * i))
        y = termino - c          # Suma el error acumulado anterior
        t = suma + y            # Intenta acumular el valor
        c = (t - suma) - y      # Calcula el error real que se ha perdido por redondeo
        suma = t
    return suma

res_ord = suma_ordinaria(N)
res_kah = suma_kahan(N)

print(f"Valor Teórico Exacto: {valor_teorico:.16f}")
print(f"Suma Ordinaria (32b): {res_ord:.16f} | Error absoluto: {abs(valor_teorico - res_ord):.2e}")
print(f"Suma de Kahan (32b): {res_kah:.16f} | Error absoluto: {abs(valor_teorico - res_kah):.2e}")
```

La Suma de Kahan reduce drásticamente el error de redondeo acumulado, acercándose a la precisión teórica con un coste computacional mínimo.

6. Ejercicios Resueltos

Ejercicio 1

Determinar si la serie $\sum_{n=1}^{\infty} \frac{n^2}{2^n}$ es convergente o divergente.

Solución utilizando el Criterio del Cociente (D'Alembert): El término general es $a_n = \frac{n^2}{2^n}$, que es positivo para todo $n \geq 1$. Calculamos el término siguiente:

$$a_{n+1} = \frac{(n+1)^2}{2^{n+1}}$$

Ahora hallamos el límite del cociente de los términos consecutivos:

$$L = \lim_{n \rightarrow \infty} \frac{a_{n+1}}{a_n} = \lim_{n \rightarrow \infty} \frac{\frac{(n+1)^2}{2^{n+1}}}{\frac{n^2}{2^n}}$$

Simplificamos los términos exponenciales:

$$\frac{2^n}{2^{n+1}} = \frac{2^n}{2^n \cdot 2} = \frac{1}{2}$$

Por lo tanto:

$$L = \lim_{n \rightarrow \infty} \frac{(n+1)^2}{2 \cdot n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \left(\frac{n+1}{n} \right)^2$$

Puesto que $\lim_{n \rightarrow \infty} \frac{n+1}{n} = 1$:

$$L = \frac{1}{2} \cdot 1^2 = \frac{1}{2}$$

Dado que $L = \frac{1}{2} < 1$, por el criterio del cociente de D'Alembert la serie **converge**.

7. Ejercicios Propuestos

1. Determinar la convergencia o divergencia de la siguiente serie aplicando el criterio de comparación por límite o la raíz:

$$\sum_{n=1}^{\infty} \left(\frac{3n+1}{4n+3} \right)^n$$

2. Estudiar la convergencia absoluta y condicional de la serie alternada:

$$\sum_{n=1}^{\infty} \frac{(-1)^{n-1}}{\sqrt{n}}$$

3. Calcular el valor exacto de la suma de la siguiente serie geométrica:

$$\sum_{n=1}^{\infty} \frac{3 \cdot 2^{n-1}}{5^n}$$

Tema 4: Cálculo de Límites y Continuidad

El cálculo infinitesimal se sustenta sobre el concepto de límite. En la ingeniería informática, estos conceptos son esenciales para comprender la continuidad de señales físicas digitalizadas, el modelado de variables continuas en simulación física y el análisis de algoritmos de optimización o búsqueda continua de raíces.

1. Límites de Funciones Reales

Consideramos una función real de variable real $f : D \subset \mathbb{R} \rightarrow \mathbb{R}$.

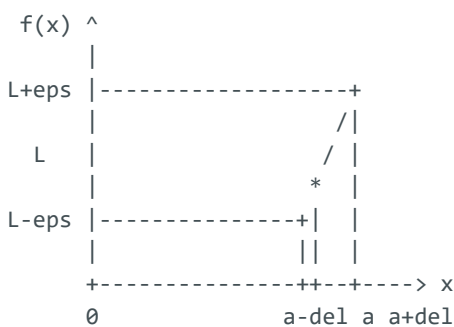
1.1 Definición Formal de Límite (Límite de Weierstrass o definición $\varepsilon - \delta$)

Decimos que el límite de $f(x)$ cuando x tiende a a es $L \in \mathbb{R}$, y lo escribimos como:

$$\lim_{x \rightarrow a} f(x) = L$$

si para cada número real $\varepsilon > 0$, existe un número real $\delta > 0$ tal que para todo $x \in D$, si la distancia entre x y a (sin ser $x = a$) es menor que δ , entonces la distancia entre $f(x)$ y L es menor que ε :

$$\lim_{x \rightarrow a} f(x) = L \iff \forall \varepsilon > 0, \exists \delta > 0 \text{ tal que } \forall x \in D, 0 < |x - a| < \delta \implies |f(x) - L| < \varepsilon$$



1.2 Límites Laterales

- Límite por la izquierda:** x se aproxima a a desde valores menores que a ($x < a$):

$$\lim_{x \rightarrow a^-} f(x) = L_1$$

- Límite por la derecha:** x se aproxima a a desde valores mayores que a ($x > a$):

$$\lim_{x \rightarrow a^+} f(x) = L_2$$

[!NOTE] El límite $\lim_{x \rightarrow a} f(x)$ existe si y solo si existen ambos límites laterales y son iguales:

$$\lim_{x \rightarrow a} f(x) = L \Leftrightarrow \lim_{x \rightarrow a^-} f(x) = \lim_{x \rightarrow a^+} f(x) = L$$

2. Continuidad de Funciones

Una función $f(x)$ es **continua en un punto** $a \in D$ si satisface las tres condiciones siguientes:

1. Existe $f(a)$ (la función está definida en a).
2. Existe el límite $\lim_{x \rightarrow a} f(x)$.
3. El valor del límite coincide con el valor de la función:

$$\lim_{x \rightarrow a} f(x) = f(a)$$

Una función es **continua en un intervalo abierto** (a, b) si es continua en cada uno de sus puntos. Es **continua en un intervalo cerrado** $[a, b]$ si es continua en (a, b) , continua por la derecha en a ($\lim_{x \rightarrow a^+} f(x) = f(a)$) y continua por la izquierda en b ($\lim_{x \rightarrow b^-} f(x) = f(b)$).

Tipos de Discontinuidad

Si no se cumple la definición de continuidad, el punto a se cataloga como una **discontinuidad**:

1. **Evitable**: Existe $\lim_{x \rightarrow a} f(x) = L$, pero $f(a) \neq L$ o $f(a)$ no está definido.
2. **Inevitables o Esenciales**:
 - **De salto finito**: Los límites laterales existen pero son distintos:

$$|\lim_{x \rightarrow a^-} f(x) - \lim_{x \rightarrow a^+} f(x)| = K < \infty$$

- **De salto infinito**: Al menos uno de los límites laterales es $+\infty$ o $-\infty$.

3. Teoremas Fundamentales de Continuidad

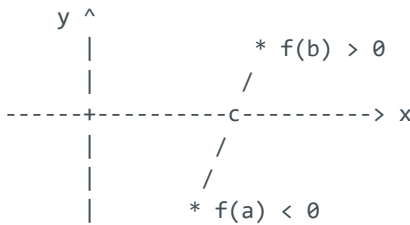
Estos teoremas proporcionan garantías analíticas cruciales para la resolución de ecuaciones y problemas de optimización.

3.1 Teorema de Bolzano

Sea $f(x)$ una función continua en el intervalo cerrado $[a, b]$. Si la función toma valores de signos opuestos en los extremos del intervalo (es decir, $f(a) \cdot f(b) < 0$), entonces existe al menos un punto $c \in (a, b)$ tal que:

$$f(c) = 0$$

Interpretación geométrica: Si una curva continua empieza por debajo del eje X y termina por encima (o viceversa), necesariamente tiene que cruzar el eje X en algún punto intermedio.



3.2 Teorema del Valor Intermedio (Darboux)

Si $f(x)$ es continua en $[a, b]$, entonces toma todos los valores intermedios entre $f(a)$ y $f(b)$.

3.3 Teorema de Weierstrass

Si $f(x)$ es una función continua en un intervalo cerrado y acotado $[a, b]$, entonces $f(x)$ alcanza sus valores máximo y mínimo absolutos en ese intervalo. Es decir, existen $x_1, x_2 \in [a, b]$ tales que:

$$f(x_1) \leq f(x) \leq f(x_2) \quad \forall x \in [a, b]$$

4. El Toque Informático

4.1 Búsqueda de Raíces por Bipartición (Bisección)

El Teorema de Bolzano garantiza la existencia de una raíz en un intervalo $[a, b]$ si $f(a) \cdot f(b) < 0$. Este hecho analítico da origen al **Algoritmo de Bisección**, un método numérico robusto para resolver ecuaciones de la forma $f(x) = 0$.

El algoritmo divide repetidamente a la mitad el intervalo de búsqueda:

1. Calcular el punto medio $m = \frac{a+b}{2}$.
2. Si $f(m) \approx 0$ o la longitud del intervalo $(b - a)$ es menor que una tolerancia predefinida, detener la búsqueda y retornar m .
3. Evaluar el signo de $f(m)$:
 - Si $f(a) \cdot f(m) < 0$, la raíz está en $[a, m] \Rightarrow b = m$.
 - Si $f(m) \cdot f(b) < 0$, la raíz está en $[m, b] \Rightarrow a = m$.
4. Repetir.

Complejidad y Robustez

- **Complejidad temporal:** En cada paso el intervalo de búsqueda se reduce a la mitad. Tras k iteraciones, la amplitud del intervalo es $\frac{b-a}{2^k}$. Para lograr una precisión ε , requerimos un número máximo de iteraciones:

$$k = \lceil \log_2 \left(\frac{b-a}{\varepsilon} \right) \rceil$$

Esto confiere al algoritmo una **complejidad temporal de** $O(\log(1/\varepsilon))$, análoga a la búsqueda binaria en arrays.

- **Parada robusta:** Para evitar bucles infinitos causados por imprecisiones de coma flotante en funciones muy empinadas o planas, la condición de parada debe comprobar la amplitud del intervalo $|b - a| < \varepsilon$ y un límite de iteraciones.

A continuación, implementamos el algoritmo de bisección en Python para hallar la raíz de $f(x) = x^3 - x - 2$ en el intervalo $[1, 2]$.

```
import numpy as np

def f(x):
    return x**3 - x - 2

def biseccion(func, a, b, tol=1e-7, max_iter=100):
    # Validamos Bolzano
    if func(a) * func(b) >= 0:
        raise ValueError("El teorema de Bolzano no garantiza la raíz en este intervalo.")

    for iteracion in range(1, max_iter + 1):
        m = a + (b - a) / 2.0 # Evitamos cancelaciones catastróficas frente a (a+b)/2
        fm = func(m)

        # Criterios de parada
        if abs(fm) < tol or (b - a) / 2.0 < tol:
            return m, iteracion

        if func(a) * fm < 0:
            b = m
        else:
            a = m

    return m, max_iter

# Ejecución
raiz, iters = biseccion(f, 1.0, 2.0)
print(f"Raíz aproximada: {raiz:.8f}")
print(f"Número de iteraciones: {iters}")
print(f"Validación f(raiz): {f(raiz):.2e}")
```

5. Ejercicios Resueltos

Ejercicio 1

Demostrar que la ecuación $x^5 - 3x - 1 = 0$ tiene al menos una solución en el intervalo $[1, 2]$.

Solución utilizando el Teorema de Bolzano:

1. Definimos la función $f(x) = x^5 - 3x - 1$.
 2. $f(x)$ es una función polinómica. Por tanto, es continua en todo su dominio $\mathbb{R}$ y, por consiguiente, continua en el intervalo cerrado $[1, 2]$.
 3. Evaluamos los valores de la función en los extremos del intervalo:
 - $f(1) = 1^5 - 3(1) - 1 = 1 - 3 - 1 = -3$ (< 0).
 - $f(2) = 2^5 - 3(2) - 1 = 32 - 6 - 1 = 25$ (> 0).
 4. Puesto que $f(1) < 0$ y $f(2) > 0$, el producto $f(1) \cdot f(2) < 0$.
 5. Al cumplirse las hipótesis del Teorema de Bolzano, existe al menos un punto $c \in (1, 2)$ tal que $f(c) = 0$.
 6. Esto demuestra formalmente que existe al menos una solución de la ecuación en el intervalo $[1, 2]$.
-

6. Ejercicios Propuestos

1. Estudiar la continuidad de la función dada por tramos y clasificar sus discontinuidades:

$$f(x) = \begin{cases} \frac{x^2-1}{x-1} & \text{si } x < 1 \\ 3 & \text{si } x = 1 \\ x^2 + 1 & \text{si } x > 1 \end{cases}$$

2. Demostrar que las gráficas de las funciones $g(x) = e^x$ y $h(x) = x^2 + 2$ se cortan en al menos un punto del intervalo $[-2, 0]$.
3. Determinar la cantidad mínima de iteraciones del algoritmo de bisección requeridas para aproximar la raíz de una función en el intervalo $[0, 4]$ con un error de precisión menor que 10^{-6} .

Tema 5: Derivación y Aplicaciones

La derivación mide la tasa de variación instantánea de una función. En la ingeniería informática, las derivadas son el núcleo de los algoritmos de optimización (Machine Learning y Redes Neuronales), la simulación física en videojuegos, el procesamiento digital de imágenes (detección de bordes) y el análisis de algoritmos de control numérico.

1. Concepto e Interpretación de la Derivada

Dada una función $f(x)$ definida en un intervalo abierto que contiene a a , definimos la **derivada** de f en el punto a , denotada por $f'(a)$, como el límite:

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$$

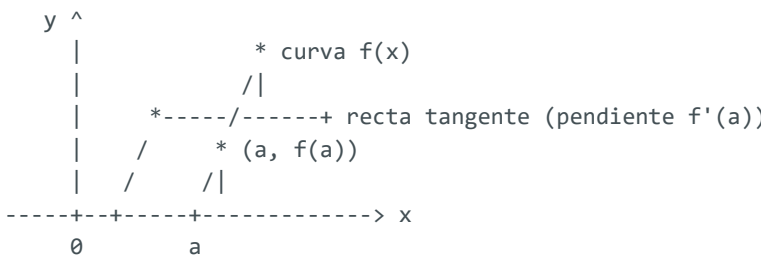
si este límite existe y es finito.

Interpretación Geométrica

La derivada $f'(a)$ representa la **pendiente de la recta tangente** a la curva $y = f(x)$ en el punto $(a, f(a))$.

La ecuación de la recta tangente en dicho punto viene dada por:

$$y - f(a) = f'(a)(x - a)$$



2. Reglas de Derivación Clásicas

Para derivar funciones complejas de forma sistemática sin recurrir a la definición por límite, empleamos las reglas algebraicas básicas y la **Regla de la Cadena** para la composición de funciones:

- Derivada de una constante:** $(c)' = 0$
- Regla de la potencia:** $(x^n)' = nx^{n-1}$

- **Derivada de la suma/resta:** $(u \pm v)' = u' \pm v'$
- **Regla del producto:** $(u \cdot v)' = u'v + uv'$
- **Regla del cociente:** $\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$
- **Regla de la Cadena (composición de funciones):** Si $y = (f \circ g)(x) = f(g(x))$, entonces:

$$(f(g(x)))' = f'(g(x)) \cdot g'(x)$$

3. Teoremas Fundamentales del Valor Medio

Estos teoremas establecen conexiones entre los valores de una función y los valores de sus derivadas en intervalos específicos.

3.1 Teorema de Rolle

Sea $f(x)$ una función continua en $[a, b]$ y derivable en (a, b) . Si $f(a) = f(b)$, entonces existe al menos un punto $c \in (a, b)$ tal que:

$$f'(c) = 0$$

Interpretación: Si una función tiene la misma altura al inicio y al final de un intervalo, debe haber un punto donde su tangente sea completamente horizontal.

3.2 Teorema del Valor Medio de Lagrange

Sea $f(x)$ continua en $[a, b]$ y derivable en (a, b) . Entonces existe al menos un punto $c \in (a, b)$ tal que:

$$f'(c) = \frac{f(b) - f(a)}{b - a}$$

Interpretación: Existe al menos un punto donde la tasa de cambio instantánea (pendiente de la tangente) es igual a la tasa de cambio promedio a lo largo de todo el intervalo (pendiente de la secante que une los extremos).

3.3 Teorema del Valor Medio de Cauchy

Sean $f(x)$ y $g(x)$ continuas en $[a, b]$ y derivables en (a, b) . Si $g'(x) \neq 0$ para todo $x \in (a, b)$, entonces existe al menos un punto $c \in (a, b)$ tal que:

$$\frac{f'(c)}{g'(c)} = \frac{f(b) - f(a)}{g(b) - g(a)}$$

3.4 Regla de L'Hôpital

Consecuencia directa del Teorema de Cauchy, se emplea para resolver límites indeterminados de tipo $\frac{0}{0}$ o $\frac{\infty}{\infty}$. Si $\lim_{x \rightarrow a} \frac{f(x)}{g(x)}$ presenta una indeterminación de tipo $\frac{0}{0}$ o $\frac{\infty}{\infty}$, y f, g son derivables en las cercanías de a con $g'(x) \neq 0$:

$$\lim_{x \rightarrow a} \frac{f(x)}{g(x)} = \lim_{x \rightarrow a} \frac{f'(x)}{g'(x)}$$

siempre que este último límite exista o sea infinito.

4. Extremos Locales y Optimización

El análisis de derivadas permite localizar los valores máximos y mínimos de una función real.

Puntos Críticos

Un punto $x = c$ en el dominio de f es un **punto crítico** si:

$$f'(c) = 0 \quad \text{o} \quad f'(c) \text{ no existe}$$

Criterio de la Segunda Derivada para Extremos Locales

Sea $f(x)$ una función dos veces derivable en un punto crítico c donde $f'(c) = 0$:

- Si $f''(c) > 0 \Rightarrow x = c$ es un **mínimo local**.
- Si $f''(c) < 0 \Rightarrow x = c$ es un **máximo local**.
- Si $f''(c) = 0$, el criterio no decide (se deben evaluar derivadas de orden superior).

5. El Toque Informático

5.1 Descenso de Gradiente (Gradient Descent)

En inteligencia artificial y aprendizaje automático, los problemas de entrenamiento (como entrenar redes neuronales o ajustar una regresión lineal) consisten en minimizar una función de coste $J(\theta)$ que mide el error del modelo.

El **Descenso de Gradiente** es un algoritmo de optimización iterativo basado en derivadas. Para una función de una sola variable $f(x)$, si queremos encontrar su mínimo local, empezamos en una estimación inicial x_0 y nos desplazamos en la **dirección opuesta a la derivada** (la derivada apunta en la dirección de máximo crecimiento; por tanto, restar la derivada nos hace descender hacia el mínimo):

$$x_{k+1} = x_k - \alpha \cdot f'(x_k)$$

donde $\alpha > 0$ es el **hiperparámetro de tasa de aprendizaje** (learning rate).

- Si α es muy pequeño, la convergencia será extremadamente lenta.
- Si α es muy grande, el algoritmo puede oscilar y divergir del mínimo.

A continuación, implementamos el algoritmo de descenso de gradiente en Python para encontrar el mínimo de la función cuadrática:

$$f(x) = x^2 - 4x + 5$$

Su derivada analítica es $f'(x) = 2x - 4$. Su mínimo exacto está en $x = 2$ (donde $f'(2) = 0$).

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x**2 - 4*x + 5

def df(x):
    return 2*x - 4

def descenso_gradiente(x_inicial, learning_rate=0.1, epochs=20, tol=1e-5):
    historial_x = [x_inicial]
    historial_y = [f(x_inicial)]

    x = x_inicial
    for epoch in range(epochs):
        gradiente = df(x)

        # Condición de parada si el gradiente es casi cero
        if abs(gradiente) < tol:
            break

        # Paso de actualización del descenso de gradiente
        x = x - learning_rate * gradiente

        historial_x.append(x)
        historial_y.append(f(x))

    return x, historial_x, historial_y

# Ejecución
x_opt, hist_x, hist_y = descenso_gradiente(x_inicial=0.0, learning_rate=0.25)

print(f"Mínimo aproximado encontrado en x = {x_opt:.6f}")
print(f"Valor mínimo de la función f(x) = {f(x_opt):.6f}")

# Graficar la trayectoria del descenso
x_val = np.linspace(-1, 5, 200)
plt.figure(figsize=(9, 5))
plt.plot(x_val, f(x_val), label='f(x) = x^2 - 4x + 5', color='blue')
plt.scatter(hist_x, hist_y, color='red', zorder=3, label='Iteraciones')
plt.plot(hist_x, hist_y, color='red', linestyle='--', alpha=0.6)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Optimización mediante Descenso de Gradiente')
plt.legend()
plt.grid(True)
plt.show()
```

6. Ejercicios Resueltos

Ejercicio 1

Encontrar las dimensiones de una caja rectangular con base cuadrada abierta por arriba que tenga un volumen de 4 m^3 y que minimice la cantidad de material empleado para su construcción (área superficial).

Solución utilizando Optimización por Derivadas:

1. **Plantear las variables:**

- Lado de la base cuadrada: x ($x > 0$).
- Altura de la caja: h ($h > 0$).

2. **Relación de volumen:**

$$V = x^2 \cdot h = 4 \implies h = \frac{4}{x^2}$$

3. **Función a optimizar (Área superficial):** La caja no tiene tapa superior, por tanto:

$$A(x, h) = \text{Área base} + 4 \cdot \text{Área caras laterales} = x^2 + 4xh$$

4. **Sustituir h para obtener una función en una sola variable x :**

$$A(x) = x^2 + 4x\left(\frac{4}{x^2}\right) = x^2 + \frac{16}{x}$$

5. **Calcular la primera derivada y buscar puntos críticos:**

$$A'(x) = 2x - \frac{16}{x^2}$$

Para encontrar los puntos críticos, igualamos a cero:

$$2x - \frac{16}{x^2} = 0 \implies 2x^3 = 16 \implies x^3 = 8 \implies x = 2$$

6. **Validar si es un mínimo mediante la segunda derivada:**

$$A''(x) = 2 + \frac{32}{x^3}$$

Evaluyendo en $x = 2$:

$$A''(2) = 2 + \frac{32}{8} = 2 + 4 = 6 > 0$$

Puesto que $A''(2) > 0$, la dimensión $x = 2 \text{ m}$ es un mínimo local.

7. **Calcular la altura h :**

$$h = \frac{4}{2^2} = 1 \text{ m}$$

8. **Dimensiones óptimas:** La base debe medir $2 \times 2 \text{ m}$ y la altura debe ser de 1 m .

7. Ejercicios Propuestos

1. Calcular aplicando la regla de L'Hôpital el siguiente límite indeterminado:

$$\lim_{x \rightarrow 0} \frac{x - \sin x}{x^3}$$

2. Estudiar los intervalos de crecimiento, decrecimiento y extremos locales de la función:

$$f(x) = xe^{-x}$$

3. Demostrar que la ecuación polinómica $3x^5 + 15x - 8 = 0$ tiene exactamente una raíz real. (*Pista: Usa primero Bolzano para asegurar la existencia, y luego Rolle para probar por contradicción la unicidad de la raíz*).

Tema 6: Polinomio de Taylor

Las funciones no polinómicas como las trigonométricas, exponenciales o logarítmicas (denominadas funciones trascendentes) son difíciles de evaluar directamente a nivel de hardware. El Polinomio de Taylor resuelve este problema transformando estas funciones complejas en sumas de polinomios ordinarios en el entorno de un punto. En la informática, esta técnica es la base del diseño de chips aritméticos, bibliotecas matemáticas estándar (`math.h`, `math` en Python) y métodos de diferencias finitas en modelado 3D.

1. Concepto y Construcción de la Aproximación Local

Dada una función $f(x)$ que es derivable n veces en un punto a , queremos encontrar un polinomio $P_n(x)$ de grado n tal que su comportamiento local en las cercanías del punto a sea lo más parecido posible al de $f(x)$.

Para lograr la mejor aproximación local, imponemos la condición de que el polinomio $P_n(x)$ y sus primeras n derivadas coincidan exactamente con la función $f(x)$ y sus primeras n derivadas en el punto $x = a$:

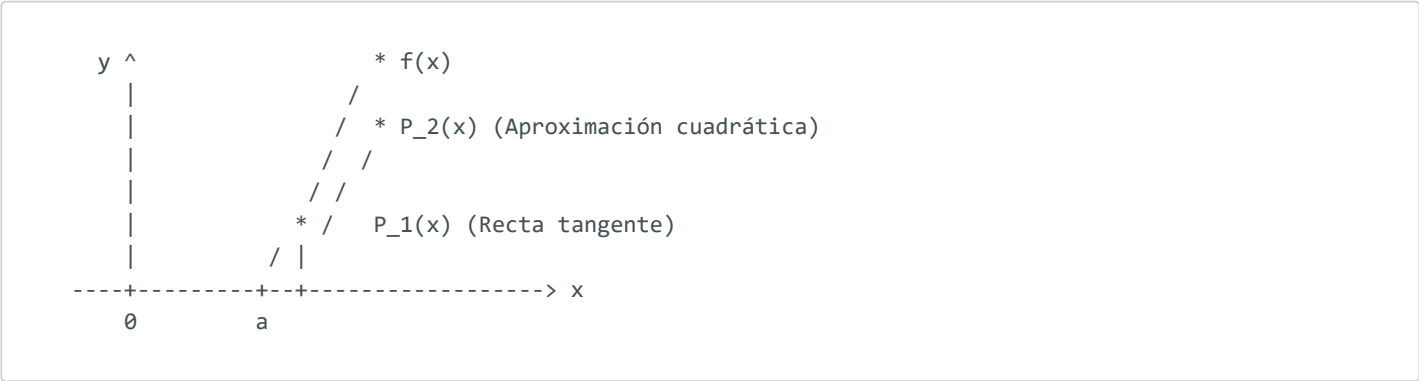
$$P_n(a) = f(a), \quad P'_n(a) = f'(a), \quad P''_n(a) = f''(a), \quad \dots, \quad P_n^{(n)}(a) = f^{(n)}(a)$$

Bajo estas condiciones, se deduce algebraicamente la fórmula del **Polinomio de Taylor de grado n para $f(x)$ centrado en a** :

$$P_n(x) = f(a) + f'(a)(x - a) + \frac{f''(a)}{2!}(x - a)^2 + \frac{f'''(a)}{3!}(x - a)^3 + \dots + \frac{f^{(n)}(a)}{n!}(x - a)^n$$

Lo cual se resume en notación sumatoria como:

$$P_n(x) = \sum_{k=0}^n \frac{f^{(k)}(a)}{k!}(x - a)^k \quad (\text{definiendo } 0! = 1 \text{ y } f^{(0)}(a) = f(a))$$



Polinomio de Maclaurin

Cuando el punto de aproximación es el origen, $a = 0$, el polinomio se denomina **Polinomio de Maclaurin**:

$$P_n(x) = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 + \dots + \frac{f^{(n)}(0)}{n!}x^n$$

2. Desarrollos de Maclaurin Elementales Importantes

A continuación se presentan los polinomios de aproximación de Maclaurin ($a = 0$) fundamentales:

Función $f(x)$	Término general del desarrollo de Maclaurin	Intervalo de validez
Exponencial: e^x	$\sum_{k=0}^n \frac{x^k}{k!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$	$x \in \mathbb{R}$
Seno: $\sin x$	$\sum_{k=0}^n \frac{(-1)^k x^{2k+1}}{(2k+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$	$x \in \mathbb{R}$
Coseno: $\cos x$	$\sum_{k=0}^n \frac{(-1)^k x^{2k}}{(2k)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots$	$x \in \mathbb{R}$
Logaritmo: $\ln(1+x)$	$\sum_{k=1}^n \frac{(-1)^{k-1} x^k}{k} = x - \frac{x^2}{2} + \frac{x^3}{3} - \dots$	$x \in (-1, 1]$

3. El Resto de la Aproximación y Estimación del Error

La aproximación polinómica comete un error. Definimos el **Resto de Taylor de orden n** , denotado por $R_n(x)$, como la diferencia exacta entre la función real y su aproximación polinómica:

$$f(x) = P_n(x) + R_n(x) \implies R_n(x) = f(x) - P_n(x)$$

Resto de Lagrange

Si la función $f(x)$ es derivable $n + 1$ veces en un intervalo abierto que contiene a a y x , entonces existe un punto intermedio c estrictamente entre a y x tal que el resto viene dado por:

$$R_n(x) = \frac{f^{(n+1)}(c)}{(n+1)!}(x-a)^{n+1}$$

Cota de Error

Para estimar el error absoluto cometido en una aproximación local dentro de un intervalo, buscamos una cota superior para la derivada de orden $n + 1$:

$$|R_n(x)| \leq \frac{M}{(n+1)!} |x-a|^{n+1} \quad \text{donde } M \geq \max_{t \in [a,x]} |f^{(n+1)}(t)|$$

4. El Toque Informático

¿Cómo calculan los ordenadores funciones trascendentes?

Una CPU o GPU no dispone de un mecanismo directo de hardware para resolver $\cos(x)$. En su lugar, utilizan algoritmos de aproximación polinómica optimizados de Taylor-Maclaurin, el método CORDIC o aproximaciones de Chebyshev combinados con **reducción de rango**.

Por ejemplo, para evaluar $\cos(x)$ para cualquier $x \in \mathbb{R}$:

1. **Reducción de rango:** Aprovechando la periodicidad del coseno, se mapea x al intervalo básico $[-\pi, \pi]$ o $[0, \pi/2]$.
2. **Evaluación polinómica:** Se calcula el polinomio de Taylor de grado truncado adecuado a ese intervalo para garantizar un error inferior a la precisión de coma flotante de la máquina (por ejemplo, 10^{-16} para una variable `double` de 64 bits).

A continuación, implementamos una aproximación de $\cos(x)$ usando Maclaurin en Python, controlando de manera dinámica el número de términos para observar cómo se reduce el error frente a la función estándar de la librería nativa de Python `math`.

```
import math
import numpy as np
import matplotlib.pyplot as plt

def aproximar_coseno(x, n_terminos):
    # Reducción de rango: mapeamos x a [-pi, pi]
    x_reducido = x % (2 * math.pi)
    if x_reducido > math.pi:
        x_reducido -= 2 * math.pi

    suma = 0.0
    for k in range(n_terminos):
        coeficiente = (-1)**k
        numerador = x_reducido ** (2*k)
        denominador = math.factorial(2*k)

        suma += coeficiente * (numerador / denominador)

    return suma

# Ejemplo numérico
x_prueba = 10.5 # Aproximadamente 3.34 radianes
exacto = math.cos(x_prueba)
```

```

print(f"Cálculo de cos({x_prueba})")
print(f"Valor exacto (math.cos): {exacto:.16f}\n")

print(f"{'Términos':<10}{ 'Aproximación':<20}{ 'Error Absoluto':<20}")
for n in range(1, 10):
    aprox = aproximar_coseno(x_prueba, n)
    error = abs(exacto - aprox)
    print(f"{n:<10}{aprox:<20.16f}{error:<20.2e}")

# Visualización del error conforme nos alejamos del origen para diferentes grados
x_vals = np.linspace(-6, 6, 300)
y_cos = np.cos(x_vals)

plt.figure(figsize=(9, 5))
plt.plot(x_vals, y_cos, label="cos(x) real", color="black", linewidth=2)

for n in [2, 4, 6]:
    # Para graficar, usamos la fórmula de Taylor directamente sin reducción de rango
    y_taylor = np.zeros_like(x_vals)
    for k in range(n):
        y_taylor += ((-1)**k) * (x_vals**(2*k)) / math.factorial(2*k)
    plt.plot(x_vals, y_taylor, label=f"Taylor Grado {2*n-2}", linestyle="--")

plt.ylim(-2, 2)
plt.title("Aproximación de cos(x) mediante Polinomios de Taylor en el origen")
plt.xlabel("x (radianes)")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

```

5. Ejercicios Resueltos

Ejercicio 1

Aproximar el número $e^{0.1}$ usando el polinomio de Maclaurin de grado 3 de $f(x) = e^x$, y acotar el error de la aproximación usando el resto de Lagrange.

Solución:

- Desarrollo de Maclaurin de $f(x) = e^x$ de grado 3:** El polinomio es:

$$P_3(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{6}$$

- Evaluar en $x = 0.1$:**

$$P_3(0.1) = 1 + 0.1 + \frac{(0.1)^2}{2} + \frac{(0.1)^3}{6} = 1 + 0.1 + 0.005 + 0.00016667 = 1.10516667$$

- Acotar el error cometido utilizando el Resto de Lagrange:** El resto $R_3(x)$ para $n = 3$ es:

$$R_3(x) = \frac{f^{(4)}(c)}{4!}x^4 = \frac{e^c}{24}x^4 \quad \text{donde } c \in (0, 0.1)$$

Como $c \in (0, 0.1)$, sabemos que e^c es estrictamente creciente. Por tanto, podemos acotar superiormente $e^c < e^{0.1} < e^1 < 3$. Sustituyendo el valor $x = 0.1$:

$$|R_3(0.1)| \leq \frac{3}{24}(0.1)^4 = 0.125 \cdot 10^{-4} = 1.25 \cdot 10^{-5}$$

El valor aproximado calculado es 1.10516667, con un error absoluto garantizado menor que 0.0000125. (De hecho, el valor real es $e^{0.1} \approx 1.105170918$, con un error real de $\approx 4.25 \cdot 10^{-6}$, lo cual cumple holgadamente con la cota teórica calculada).

6. Ejercicios Propuestos

1. Hallar el polinomio de Taylor de grado 3 de la función $f(x) = \ln(x)$ centrado en el punto $a = 1$.
2. Utilizar el polinomio de Maclaurin de grado 2 para aproximar $\sqrt{1.2}$ usando la función generalizada de potencia $(1+x)^{1/2}$, y estimar una cota superior para el error absoluto.
3. Calcular los límites siguientes utilizando desarrollos de Taylor de Maclaurin adecuados:

$$\lim_{x \rightarrow 0} \frac{\cos x - 1 + \frac{x^2}{2}}{x^4}$$

Tema 7: Integración

La integración es la operación inversa de la derivación. Mientras que la derivada mide tasas de cambio local, la integral acumula variaciones continuas para obtener totales (áreas, volúmenes, acumulaciones temporales). En ingeniería informática, la integración es fundamental para el procesamiento digital de señales, el análisis de datos de probabilidad continua (Machine Learning y Ciencia de Datos), la visión artificial y la simulación física continua.

1. Integral Indefinida y Métodos de Integración

Dada una función $f(x)$, llamamos **primitiva** o **antiderivada** a una función $F(x)$ tal que:

$$F'(x) = f(x)$$

La **integral indefinida** representa el conjunto de todas las primitivas de $f(x)$ y se denota como:

$$\int f(x) dx = F(x) + C$$

donde $C \in \mathbb{R}$ es la constante de integración.

Métodos Clásicos de Integración

Para calcular primitivas de forma analítica se utilizan cuatro métodos principales:

1. **Integración por Partes:** Derivado directamente de la regla de derivación del producto.

$$\int u dv = u \cdot v - \int v du$$

2. **Integración por Cambio de Variable (Sustitución):** Basado en la regla de la cadena.

$$\int f(g(x))g'(x) dx = \int f(u) du \quad (\text{donde } u = g(x))$$

3. **Integración de Funciones Racionales:** Consiste en descomponer cocientes de polinomios $\frac{P(x)}{Q(x)}$ en fracciones simples:

- Si $\text{grado}(P) \geq \text{grado}(Q)$, primero se realiza la división polinómica.
- Si $\text{grado}(P) < \text{grado}(Q)$, se factoriza el denominador $Q(x)$ y se descompone en sumas del tipo $\frac{A}{x-a}$, $\frac{B}{(x-a)^2}$, o $\frac{Mx+N}{x^2+px+q}$.

2. Integral Definida e Integración de Riemann

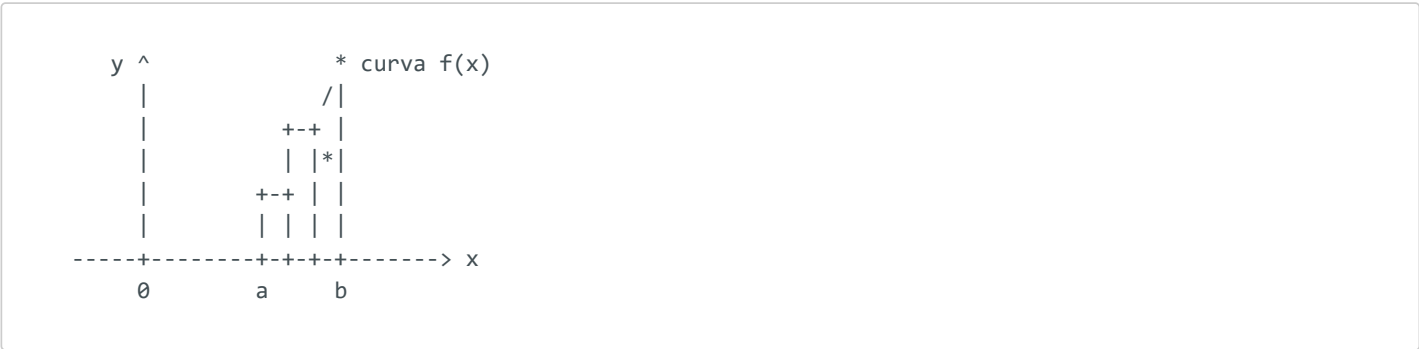
La **integral de Riemann** formaliza la definición de área bajo una curva dividiendo el dominio en rectángulos elementales.

Sumas de Riemann

Sea $f(x)$ una función acotada en $[a, b]$. Dividimos el intervalo en n subintervalos de ancho $\Delta x = \frac{b-a}{n}$. La **integral definida** se define como el límite de la suma de las áreas de estos rectángulos cuando el ancho tiende a cero (es decir, $n \rightarrow \infty$):

$$\int_a^b f(x) \, dx = \lim_{n \rightarrow \infty} \sum_{i=1}^n f(x_i^*) \Delta x$$

donde x_i^* es un punto cualquiera dentro del i -ésimo subintervalo.



3. Teoremas Fundamentales del Cálculo (TFC)

Estos teoremas conectan formalmente el cálculo diferencial con el cálculo integral.

Primer Teorema Fundamental del Cálculo

Sea $f(x)$ una función continua en el intervalo $[a, b]$. Definimos la función área (o función integral) $F(x)$ en $[a, b]$ como:

$$F(x) = \int_a^x f(t) \, dt$$

Entonces, $F(x)$ es derivable en (a, b) y se cumple que:

$$F'(x) = f(x)$$

Consecuencia: La derivación y la integración son operaciones recíprocas.

Segundo Teorema Fundamental (Regla de Barrow)

Si $f(x)$ es continua en $[a, b]$ y $F(x)$ es cualquier primitiva de $f(x)$ en dicho intervalo (es decir, $F'(x) = f(x)$):

$$\int_a^b f(x) dx = F(b) - F(a) = [F(x)]_a^b$$

4. Integrales Impropias

Ampliamos la definición de integral definida cuando el intervalo de integración no es acotado o la función presenta singularidades.

4.1 Integrales de Primera Especie (Límites infinitos)

El intervalo de integración se extiende al infinito:

$$\int_a^{\infty} f(x) dx = \lim_{t \rightarrow \infty} \int_a^t f(x) dx$$

La integral se dice **convergente** si el límite existe y es finito; en caso contrario, se dice **divergente**.

4.2 Integrales de Segunda Especie (Funciones no acotadas)

La función presenta una asíntota vertical (tiende a infinito) en algún punto del intervalo de integración:

- Si $f(x)$ es discontinua en el extremo derecho b :

$$\int_a^b f(x) dx = \lim_{t \rightarrow b^-} \int_a^t f(x) dx$$

5. El Toque Informático

Integración en Probabilidad Continua

En ciencia de datos, machine learning y procesamiento digital de imágenes, es común trabajar con variables aleatorias continuas. La probabilidad de que una variable aleatoria X tome un valor dentro de un intervalo $[a, b]$ es el área bajo su **función de densidad de probabilidad** (PDF):

$$P(a \leq X \leq b) = \int_a^b f(x) dx$$

El ejemplo más emblemático es la **distribución normal (gaussiana)** con media $\mu = 0$ y desviación estándar $\sigma = 1$:

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

Esta función no tiene una primitiva expresable mediante funciones elementales (es decir, la integral indefinida $\int e^{-x^2} dx$ no se puede escribir con polinomios, trigonométricas o logaritmos simples). Por tanto, para calcular esta integral definida en un ordenador se recurre a aproximaciones numéricas o funciones especiales implementadas en librerías científicas (**scipy**).

A continuación, implementamos en Python un script que calcula la probabilidad acumulada de una distribución normal usando integración simbólica/numérica con **sympy** y **scipy.integrate**.

```
import numpy as np
import sympy as sp
from scipy.integrate import quad

# 1. Enfoque Simbólico con SymPy
def probabilidad_normal_simbolica(a, b):
    x = sp.Symbol('x')
    # Definimos la PDF de la normal estándar
    pdf = (1 / sp.sqrt(2 * sp.pi)) * sp.exp(-x**2 / 2)
    # Calculamos la integral definida simbólicamente
    probabilidad = sp.integrate(pdf, (x, a, b))
    return probabilidad.evalf()

# 2. Enfoque Numérico Rápido con SciPy
def probabilidad_normal_numerica(a, b):
    # PDF normal estándar definida en Python
    pdf = lambda x: (1.0 / np.sqrt(2.0 * np.pi)) * np.exp(-x**2 / 2.0)
    # quad realiza la integración de Riemann numérica adaptativa de Gauss-Kronrod
    resultado, error_estimado = quad(pdf, a, b)
    return resultado, error_estimado

# Queremos calcular la probabilidad dentro de "una desviación estándar" [-1, 1]
# Teóricamente, esto debería ser aproximadamente 68.27% (regla empírica)
lim_inf = -1.0
lim_sup = 1.0

prob_simb = probabilidad_normal_simbolica(lim_inf, lim_sup)
prob_num, error_num = probabilidad_normal_numerica(lim_inf, lim_sup)

print(f"Intervalo de integración: [{lim_inf}, {lim_sup}]")
print(f"Probabilidad Simbólica (SymPy): {prob_simb}")
print(f"Probabilidad Numérica (SciPy): {prob_num:.8f} (Error estimado: {error_num:.2e})")
```

6. Ejercicios Resueltos

Ejercicio 1

Calcular la integral definida:

$$\int_0^1 x e^x dx$$

Solución usando la Regla de Barrow e Integración por Partes:

1. **Encontrar la primitiva analítica (integral indefinida):**

$$\int x e^x dx$$

Aplicamos integración por partes con la fórmula $\int u dv = u \cdot v - \int v du$:

- Elegimos $u = x \Rightarrow du = dx$
- Elegimos $dv = e^x dx \Rightarrow v = e^x$ Sustituyendo en la fórmula:

$$\int x e^x dx = x e^x - \int e^x dx = x e^x - e^x = e^x(x - 1)$$

Nuestra primitiva es $F(x) = e^x(x - 1)$.

2. **Aplicar el Segundo Teorema Fundamental del Cálculo (Barrow):**

$$\int_0^1 x e^x dx = [e^x(x - 1)]_0^1$$

- Evaluando en el extremo superior ($x = 1$):

$$F(1) = e^1(1 - 1) = e \cdot 0 = 0$$

- Evaluando en el extremo inferior ($x = 0$):

$$F(0) = e^0(0 - 1) = 1 \cdot (-1) = -1$$

3. **Restar los valores obtenidos:**

$$\int_0^1 x e^x dx = F(1) - F(0) = 0 - (-1) = 1$$

El área exacta bajo la curva en el intervalo $[0, 1]$ es 1.

7. Ejercicios Propuestos

1. Calcular la integral indefinida mediante el método de descomposición en fracciones simples:

$$\int \frac{2x + 3}{x^2 - 3x + 2} dx$$

2. Calcular la siguiente integral definida por cambio de variable:

$$\int_0^2 x \sqrt{x^2 + 1} dx$$

3. Determinar la convergencia o divergencia de la siguiente integral impropia de primera especie:

$$\int_1^{\infty} \frac{1}{x^3} dx$$

Si es convergente, calcular su valor exacto.

Tema 8: Resolución Numérica de Ecuaciones

Encontrar las soluciones exactas a ecuaciones de la forma $f(x) = 0$ suele ser imposible de forma analítica cuando se trata de funciones no lineales complejas. El cálculo numérico proporciona algoritmos iterativos para aproximar estas raíces con la precisión deseada. Estos métodos sustentan el funcionamiento de motores de física en videojuegos, la resolución de trayectorias espaciales y la optimización de sistemas en ingeniería.

1. Planteamiento del Problema

Buscamos aproximar un número real $\alpha \in \mathbb{R}$ tal que:

$$f(\alpha) = 0$$

donde f es una función continua en un intervalo que contiene a la raíz α .

Los métodos numéricos son **iterativos**: parten de una aproximación inicial (o un intervalo) y generan una sucesión de aproximaciones $\{x_k\}_{k=0}^{\infty}$ que esperamos que converja a la raíz:

$$\lim_{k \rightarrow \infty} x_k = \alpha$$

2. Método de la Bisección (Revisión y Análisis Numérico)

Como vimos en el Tema 4, el método de la Bisección se basa en el Teorema de Bolzano.

Análisis del Error y Convergencia

Si $[a_0, b_0]$ es el intervalo inicial y denotamos por x_k el punto medio calculado en la iteración k :

- La raíz real α y la aproximación x_k están contenidas en el intervalo de la etapa k , cuyo tamaño es $\frac{b_0 - a_0}{2^k}$.
- Por tanto, el **error absoluto cometido en el paso k** , denotado por $e_k = |x_k - \alpha|$, está estrictamente acotado por:

$$e_k \leq \frac{b_0 - a_0}{2^{k+1}}$$

Este método garantiza una **convergencia lineal** con un factor de reducción de error constante de 0.5 en cada paso. Es muy robusto y siempre converge si se cumple la hipótesis de Bolzano, pero su velocidad es relativamente lenta.

3. Método de Newton-Raphson

El **Método de Newton-Raphson** es uno de los algoritmos de resolución numérica más potentes. En lugar de dividir intervalos, utiliza la información de la derivada de la función para acelerar la convergencia.

Derivación mediante Polinomio de Taylor

Si disponemos de una aproximación x_k cercana a la raíz α , aproximamos la función $f(x)$ mediante su polinomio de Taylor de primer grado (recta tangente) en el punto x_k :

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k)$$

Buscamos el punto de corte con el eje X de esta recta de aproximación, imponiendo $f(x) = 0$:

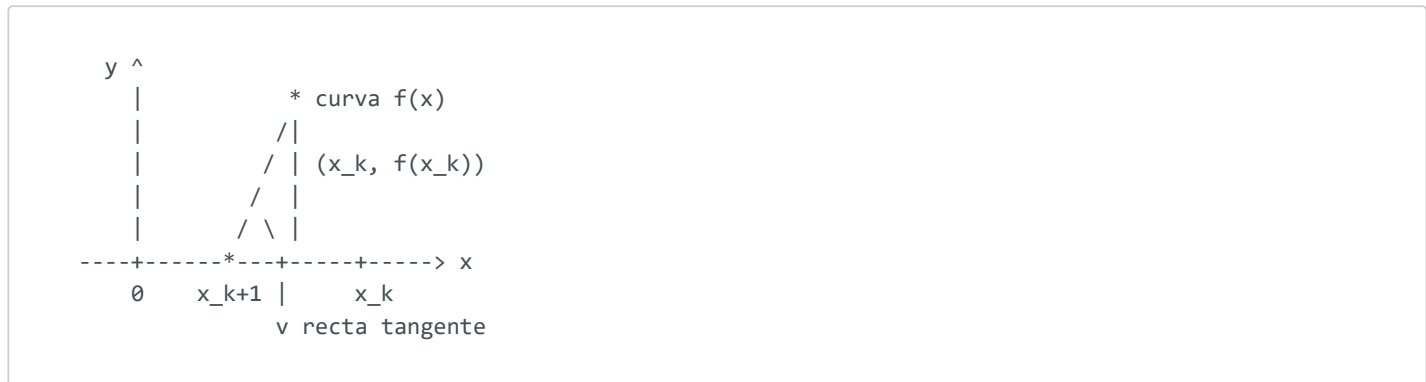
$$0 \approx f(x_k) + f'(x_k)(x - x_k) \implies x - x_k \approx -\frac{f(x_k)}{f'(x_k)}$$

Esto da origen a la **fórmula iterativa de Newton-Raphson**:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad (\text{para } f'(x_k) \neq 0)$$

Interpretación Geométrica

En cada paso k , trazamos la recta tangente a la curva $y = f(x)$ en el punto $(x_k, f(x_k))$. La intersección de esta recta tangente con el eje horizontal define la siguiente aproximación x_{k+1} .



4. Convergencia del Método de Newton-Raphson

El método de Newton-Raphson presenta propiedades de convergencia excelentes, pero también limitaciones críticas.

Convergencia Cuadrática Local

Si la función $f(x)$ es dos veces continuamente derivable, la raíz α es simple ($f'(\alpha) \neq 0$) y el valor inicial x_0 se elige suficientemente cerca de α , entonces el método converge de forma **cuadrática**:

$$\lim_{k \rightarrow \infty} \frac{|x_{k+1} - \alpha|}{|x_k - \alpha|^2} = \left| \frac{f''(\alpha)}{2f'(\alpha)} \right|$$

Significado práctico: El número de dígitos decimales de precisión correctos se **duplica** en cada iteración cuando nos encontramos cerca de la raíz.

4.2 Limitaciones e Inestabilidades

1. **Dependencia del punto de partida:** Si x_0 no está cerca de la raíz, el método puede oscilar infinitamente o divergir.
2. **Derivada nula:** Si en algún paso $f'(x_k) \approx 0$, la división por la derivada causa un error aritmético o proyecta la aproximación al infinito.
3. **Raíces múltiples:** Si la raíz tiene multiplicidad $m > 1$ (es decir, $f(\alpha) = 0$ y $f'(\alpha) = 0$), la convergencia del método de Newton se degrada de cuadrática a lineal.

5. El Toque Informático

A continuación, implementamos en Python ambos métodos (Bisección y Newton-Raphson) para resolver la ecuación:

$$f(x) = \cos(x) - x = 0$$

Su derivada es $f'(x) = -\sin(x) - 1$. Comparamos la velocidad de convergencia imprimiendo la evolución del error absoluto estimado paso a paso.

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return np.cos(x) - x

def df(x):
    return -np.sin(x) - 1.0

# 1. Implementación de Bisección
def biseccion(func, a, b, tol=1e-12, max_iter=50):
    historial = []
    if func(a) * func(b) >= 0:
        return None, historial
    for k in range(max_iter):
        x_k = a + (b - a) / 2.0
        fx = func(x_k)
        historial.append(x_k)
        if abs(fx) < tol or (b - a) / 2.0 < tol:
            break
```

```

    if func(a) * fx < 0:
        b = x_k
    else:
        a = x_k
    return x_k, historial

# 2. Implementación de Newton-Raphson
def newton_raphson(func, dfunc, x0, tol=1e-12, max_iter=50):
    historial = [x0]
    x = x0
    for k in range(max_iter):
        derivada = dfunc(x)
        if abs(derivada) < 1e-15:
            print("Advertencia: Derivada cercana a cero.")
            break
        x_siguiete = x - func(x) / derivada
        historial.append(x_siguiete)
        if abs(x_siguiete - x) < tol:
            break
        x = x_siguiete
    return x, historial

# Ejecución
raiz_exacta = 0.7390851332151606 # Número de Dottie
raiz_bis, hist_bis = biseccion(f, 0.0, 1.0)
raiz_new, hist_new = newton_raphson(f, df, 0.0)

# Comparar velocidad de convergencia (errores acumulados)
err_bis = [abs(x - raiz_exacta) for x in hist_bis]
err_new = [abs(x - raiz_exacta) for x in hist_new]

print(f"Raíz Bisección: {raiz_bis:.12f} (Iteraciones: {len(hist_bis)})")
print(f"Raíz Newton: {raiz_new:.12f} (Iteraciones: {len(hist_new)-1})")

# Gráfico de convergencia del error
plt.figure(figsize=(9, 5))
plt.plot(err_bis, label='Bisección (Convergencia Lineal)', marker='o', color='blue')
plt.plot(err_new, label='Newton-Raphson (Convergencia Cuadrática)', marker='s', color='red')
plt.yscale('log')
plt.xlabel('Iteración')
plt.ylabel('Error absoluto  $|x_k - \alpha|$ ')
plt.title('Comparación de Velocidad de Convergencia')
plt.legend()
plt.grid(True, which="both", ls="--")
plt.show()

```

La gráfica muestra cómo Newton-Raphson necesita apenas 5 iteraciones para alcanzar la precisión del límite de coma flotante de doble precisión de 64 bits, mientras que la Bisección requiere cerca de 40 iteraciones para lograr la misma precisión.

6. Ejercicios Resueltos

Ejercicio 1

Aplicar el método de Newton-Raphson para aproximar el valor de $\sqrt{2}$ mediante la resolución de la ecuación $f(x) = x^2 - 2 = 0$, comenzando con $x_0 = 1.5$. Realizar dos iteraciones de forma manual y evaluar el error cometido.

Solución:

1. Definir la función y su derivada:

$$f(x) = x^2 - 2 \Rightarrow f'(x) = 2x$$

2. Fórmula de recurrencia:

$$x_{k+1} = x_k - \frac{x_k^2 - 2}{2x_k} = x_k - \frac{x_k}{2} + \frac{1}{x_k} = \frac{x_k}{2} + \frac{1}{x_k}$$

(Esta es la conocida fórmula de Herón para la raíz cuadrada).

3. Iteración 1 ($k = 0$):

$$x_1 = \frac{1.5}{2} + \frac{1}{1.5} = 0.75 + \frac{2}{3} = 0.75 + 0.66666667 = 1.41666667$$

Error absoluto exacto ($|\sqrt{2} - x_1|$):

$$|1.41421356 - 1.41666667| = 0.00245311 \approx 2.45 \cdot 10^{-3}$$

4. Iteración 2 ($k = 1$):

$$x_2 = \frac{1.41666667}{2} + \frac{1}{1.41666667} = 0.70833333 + \frac{12}{17} = 0.70833333 + 0.70588235 = 1.41421569$$

Error absoluto exacto ($|\sqrt{2} - x_2|$):

$$|1.41421356 - 1.41421569| = 0.00000213 \approx 2.13 \cdot 10^{-6}$$

Notamos que el error disminuye drásticamente de 10^{-3} a 10^{-6} en una sola iteración, evidenciando la convergencia cuadrática del método.

7. Ejercicios Propuestos

1. Dada la ecuación $x^3 - x - 1 = 0$, aplicar el método de Newton-Raphson para realizar tres iteraciones partiendo de $x_0 = 1.5$.
2. Estudiar analíticamente el comportamiento del método de Newton-Raphson al intentar resolver la ecuación $x^{1/3} = 0$, comenzando con cualquier punto de partida $x_0 \neq 0$. ¿Qué ocurre con la convergencia? Explicar la causa.
3. Diseñar la fórmula iterativa del método de Newton-Raphson para calcular la función recíproca $\frac{1}{R}$ sin emplear operaciones de división, es decir, resolviendo $f(x) = \frac{1}{x} - R = 0$. (Esta aproximación se utiliza en hardware informático a nivel de ALU para implementar la división rápida).

Tema 9: Interpolación Polinómica

El problema de la interpolación consiste en encontrar una función sencilla (normalmente un polinomio) que pase exactamente por un conjunto dado de puntos de datos discretos. En la ingeniería informática, la interpolación es fundamental para el redimensionado de imágenes (reescalado de píxeles), la reconstrucción de señales analógicas digitalizadas, la compresión de datos y la animación 3D (curvas clave de movimiento o Splines).

1. Formulación del Problema

Disponemos de un conjunto de $n + 1$ puntos (nodos) de datos discretos:

$$(x_0, y_0), (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

donde los nodos x_i son todos distintos entre sí ($x_i \neq x_j$ para todo $i \neq j$).

Buscamos un **polinomio interpolador** $P_n(x)$ de grado menor o igual a n tal que satisfaga las condiciones de interpolación:

$$P_n(x_i) = y_i \quad \text{para todo } i = 0, 1, \dots, n$$

[!NOTE] **Teorema de Existencia y Unicidad:** Dados $n + 1$ puntos con abscisas distintas, existe un único polinomio $P_n(x)$ de grado a lo sumo n que pasa exactamente por todos ellos. Aunque usemos diferentes métodos de construcción (Lagrange, Newton, sistemas lineales), obtendremos algebraicamente el mismo polinomio.

2. Polinomio Interpolador de Lagrange

El método de Lagrange construye el polinomio como una combinación lineal de las ordenadas y_i ponderadas por **polinomios de base de Lagrange**, denotados por $L_i(x)$.

Polinomios de la Base

Para cada nodo i , definimos $L_i(x)$ como un polinomio de grado n que vale 1 en el nodo x_i y 0 en todos los demás nodos x_j ($j \neq i$):

$$L_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j} = \frac{(x - x_0)(x - x_1) \dots (x - x_{i-1})(x - x_{i+1}) \dots (x - x_n)}{(x_i - x_0)(x_i - x_1) \dots (x_i - x_{i-1})(x_i - x_{i+1}) \dots (x_i - x_n)}$$

Construcción del Polinomio

El polinomio interpolador final es:

$$P_n(x) = \sum_{i=0}^n y_i L_i(x)$$

Limitación: Si se añade un nuevo punto de datos (x_{n+1}, y_{n+1}) , todos los polinomios de la base $L_i(x)$ deben recalcularse por completo.

3. Interpolación de Newton y Diferencias Divididas

El método de Newton resuelve la rigidez de Lagrange permitiendo añadir nuevos puntos de manera incremental sin rehacer el trabajo previo.

El polinomio se escribe en la **forma de Newton**:

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_n(x - x_0)(x - x_1) \dots (x - x_{n-1})$$

Los coeficientes a_i se calculan sistemáticamente mediante **diferencias divididas**, denotadas por $f[x_0, x_1, \dots, x_k]$.

Definición de Diferencias Divididas

- Orden 0: $f[x_i] = y_i$
- Orden 1: $f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}$
- Orden general k :

$$f[x_0, x_1, \dots, x_k] = \frac{f[x_1, \dots, x_k] - f[x_0, \dots, x_{k-1}]}{x_k - x_0}$$

Los coeficientes del polinomio de Newton son precisamente las diferencias divididas de la diagonal superior:

$$a_k = f[x_0, x_1, \dots, x_k]$$

4. Análisis del Error y Fenómeno de Runge

Si los puntos provienen de una función continua $f(x)$, el error de interpolación en cualquier punto x es:

$$E(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(c)}{(n+1)!} \prod_{i=0}^n (x - x_i) \quad \text{donde } c \in (x_0, x_n)$$

El Fenómeno de Runge

A finales del siglo XIX, Carl Runge descubrió que al interpolar funciones continuas usando nodos equiespaciados, aumentar el grado del polinomio n puede hacer que el error aumente de forma ilimitada, produciendo **fuertes oscilaciones en los extremos del intervalo**. La función clásica de Runge es:

$$f(x) = \frac{1}{1 + 25x^2} \quad \text{en } [-1, 1]$$

Solución: Nodos de Chebyshev

Para minimizar la oscilación del término de error $\prod (x - x_i)$, se utilizan nodos no equiespaciados distribuidos más densamente cerca de los extremos. Estos se conocen como **nodos de Chebyshev**:

$$x_k = \cos\left(\frac{2k+1}{2n+2}\pi\right) \quad \text{para } k = 0, 1, \dots, n$$

5. El Toque Informático

A continuación, implementamos en Python el cálculo del polinomio interpolador y visualizamos el **Fenómeno de Runge** comparando la interpolación de la función de Runge con 11 nodos equiespaciados frente a 11 nodos de Chebyshev.

```
import numpy as np
import matplotlib.pyplot as plt

def funcion_runge(x):
    return 1.0 / (1.0 + 25.0 * x**2)

# Algoritmo de Interpolación de Lagrange
def interpolar_lagrange(x_nodos, y_nodos, x_eval):
    n = len(x_nodos)
    resultado = np.zeros_like(x_eval)

    for i in range(n):
        # Calculamos el polinomio base L_i(x)
        L_i = np.ones_like(x_eval)
        for j in range(n):
            if i != j:
                L_i *= (x_eval - x_nodos[j]) / (x_nodos[i] - x_nodos[j])
        resultado += y_nodos[i] * L_i

    return resultado

# Definimos los nodos de interpolación en [-1, 1]
N = 10 # Grado del polinomio n = 10 (11 nodos)

# 1. Nodos equiespaciados
nodos_eq = np.linspace(-1, 1, N + 1)
y_eq = funcion_runge(nodos_eq)
```

```
# 2. Nodos de Chebyshev
nodos_ch = np.cos((2 * np.arange(N + 1) + 1) / (2 * N + 2) * np.pi)
y_ch = funcion_runge(nodos_ch)

# Evaluación en malla densa para graficar
x_grid = np.linspace(-1, 1, 300)
y_real = funcion_runge(x_grid)
y_interp_eq = interpolator_lagrange(nodos_eq, y_eq, x_grid)
y_interp_ch = interpolator_lagrange(nodos_ch, y_ch, x_grid)

# Gráfico
plt.figure(figsize=(10, 6))
plt.plot(x_grid, y_real, label="Función de Runge real", color="black", linewidth=2)
plt.plot(x_grid, y_interp_eq, label="Interp. Equiespaciada (Runge)", linestyle="--", color="red")
plt.plot(x_grid, y_interp_ch, label="Interp. Chebyshev (Sin oscilación)", linestyle="-.", color="green")
plt.scatter(nodos_eq, y_eq, color="red", zorder=3, label="Nodos Equiespaciados")
plt.scatter(nodos_ch, y_ch, color="green", zorder=3, label="Nodos Chebyshev")

plt.ylim(-0.3, 1.3)
plt.title(f"Fenómeno de Runge vs Nodos de Chebyshev (Grado {N})")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()
```

La gráfica ilustra claramente cómo el uso de nodos equiespaciados introduce desviaciones masivas cerca de -1 y 1 (fenómeno de Runge), mientras que la distribución óptima de Chebyshev estabiliza por completo el polinomio.

6. Ejercicios Resueltos

Ejercicio 1

Obtener el polinomio interpolador en forma de Newton para los siguientes puntos: $(0, 1)$, $(1, 3)$ y $(2, 7)$.

Solución usando la Tabla de Diferencias Divididas:

1. Establecer los nodos:

- $x_0 = 0, y_0 = 1$
- $x_1 = 1, y_1 = 3$
- $x_2 = 2, y_2 = 7$

2. Construir la tabla de diferencias divididas:

- **Orden 0 (Valores de y):**

$$f[x_0] = 1, \quad f[x_1] = 3, \quad f[x_2] = 7$$

- **Orden 1:**

$$f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0} = \frac{3 - 1}{1 - 0} = 2$$

$$f[x_1, x_2] = \frac{f[x_2] - f[x_1]}{x_2 - x_1} = \frac{7 - 3}{2 - 1} = 4$$

◦ **Orden 2:**

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{4 - 2}{2 - 0} = \frac{2}{2} = 1$$

3. **Construir el polinomio en la forma de Newton:** Tomando los valores de la diagonal superior ($a_0 = 1, a_1 = 2, a_2 = 1$):

$$P_2(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1)$$

$$P_2(x) = 1 + 2(x - 0) + 1(x - 0)(x - 1) = 1 + 2x + x(x - 1) = x^2 + x + 1$$

4. **Verificación:**

- $P_2(0) = 0^2 + 0 + 1 = 1$ (Correcto)
- $P_2(1) = 1^2 + 1 + 1 = 3$ (Correcto)
- $P_2(2) = 2^2 + 2 + 1 = 7$ (Correcto)

El polinomio interpolador es $P_2(x) = x^2 + x + 1$.

7. Ejercicios Propuestos

1. Determinar el polinomio interpolador de Lagrange para los puntos:

$$(1, -1), \quad (2, 4), \quad (3, 11)$$

2. Dada la tabla de datos de un sensor de temperatura:

Tiempo t	Temperatura $T(t)$
0	20
2	22
5	28

Utilizar la interpolación de Newton para estimar la temperatura en el instante $t = 3$.

3. Demostrar que la suma de los polinomios de la base de Lagrange para cualquier conjunto de nodos es idénticamente igual a 1, es decir:

$$\sum_{i=0}^n L_i(x) = 1 \quad \forall x \in \mathbb{R}$$

(Pista: Considera qué ocurre si interpolas la función constante $f(x) = 1$).

Tema 10: Integración Numérica

La integración numérica (o cuadratura numérica) consiste en calcular una aproximación del valor de una integral definida $\int_a^b f(x) dx$ utilizando un conjunto de valores discretos de la función. En la ingeniería informática, se aplica en motores de física 2D/3D (integración de leyes de movimiento de Newton para actualizar velocidad y posición), simulación de fluidos, procesamiento de señales de audio/vídeo y cálculo probabilístico en aprendizaje automático.

1. Motivación y Fórmulas de Newton-Cotes

Existen dos razones principales para recurrir a la integración numérica:

- La función $f(x)$ carece de primitiva elemental:** Por ejemplo, $\int_0^1 e^{-x^2} dx$ o $\int_a^b \frac{\sin x}{x} dx$.
- Solo disponemos de datos muestreados:** En lugar de una expresión analítica, tenemos una tabla de valores proveniente de sensores o experimentos en intervalos discretos.

Las **fórmulas de Newton-Cotes** aproximan la integral sustituyendo la función original $f(x)$ por su polinomio interpolador $P_n(x)$ en el intervalo $[a, b]$, y calculando la integral exacta del polinomio:

$$\int_a^b f(x) dx \approx \int_a^b P_n(x) dx$$

2. La Regla del Trapecio

La regla del Trapecio aproxima la función $f(x)$ por una línea recta (polinómico de grado 1) que conecta los extremos del intervalo $(a, f(a))$ y $(b, f(b))$.

2.1 Regla del Trapecio Simple

El área bajo la línea recta es el área de un trapecio geométrico:

$$\int_a^b f(x) dx \approx \frac{b-a}{2} (f(a) + f(b))$$

El **error de truncamiento** cometido viene dado por:

$$E_T = -\frac{(b-a)^3}{12} f''(c) \quad \text{donde } c \in (a, b)$$

2.2 Regla del Trapecio Compuesta

Para mejorar la precisión, dividimos el intervalo $[a, b]$ en n subintervalos de igual ancho $h = \frac{b-a}{n}$ definidos por los nodos $x_i = a + ih$. Aplicamos la regla simple en cada subintervalo y sumamos:

$$\int_a^b f(x) dx \approx \frac{h}{2} [f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n)]$$

El **error compuesto** está acotado por:

$$E_{T,comp} = -\frac{(b-a)h^2}{12} f''(c) \Rightarrow O(h^2)$$

La precisión aumenta cuadráticamente al reducir el tamaño de paso h .

3. La Regla de Simpson (Regla de Simpson 1/3)

La regla de Simpson aproxima $f(x)$ mediante un polinomio cuadrático (parábola de grado 2) que pasa por tres puntos: los extremos del intervalo y el punto medio $m = \frac{a+b}{2}$.

3.1 Regla de Simpson Simple

$$\int_a^b f(x) dx \approx \frac{b-a}{6} [f(a) + 4f(\frac{a+b}{2}) + f(b)]$$

El **error de truncamiento** es:

$$E_S = -\frac{(b-a)^5}{2880} f^{(4)}(c) \quad \text{donde } c \in (a, b)$$

[!TIP] Dado que el error depende de la cuarta derivada $f^{(4)}(c)$, la regla de Simpson es **exacta para cualquier polinomio de grado 3 o menor**, a pesar de estar basada en una parábola cuadrática.

3.2 Regla de Simpson Compuesta

Dividimos el intervalo $[a, b]$ en un **número par** n de subintervalos de ancho $h = \frac{b-a}{n}$. Aplicando la regla simple a cada par consecutivo de intervalos resulta:

$$\int_a^b f(x) dx \approx \frac{h}{3} [f(x_0) + 4 \sum_{i=1,3,\dots}^{n-1} f(x_i) + 2 \sum_{j=2,4,\dots}^{n-2} f(x_j) + f(x_n)]$$

El **error compuesto** de Simpson está acotado por:

$$E_{S,comp} = -\frac{(b-a)h^4}{180} f^{(4)}(c) \Rightarrow O(h^4)$$

Esta tasa de convergencia de orden 4 la hace extremadamente eficiente y precisa para funciones suaves.

4. El Toque Informático

Integración Numérica en Motores de Física de Videojuegos

En el desarrollo de un motor físico (como los usados en motores de videojuegos o simuladores), debemos integrar las ecuaciones diferenciales de movimiento de Newton para actualizar el estado del juego en cada fotograma (frame rate):

$$\frac{dx}{dt} = v(t), \quad \frac{dv}{dt} = a(t) = \frac{F(t)}{m}$$

Si bien se utilizan integradores simplificados en tiempo real (como la **Integración de Euler o Verlet**), las simulaciones científicas de precisión y el análisis estadístico offline recurren a las reglas compuestas de Trapecio y Simpson para procesar grandes flujos de datos.

A continuación, implementamos en Python ambas reglas compuestas para resolver la integral de una función no elemental común en física y estadística:

$$I = \int_0^1 e^{-x^2} dx$$

Comparamos el error absoluto de ambos métodos frente a la solución exacta de alta precisión.

```
import numpy as np
import scipy.integrate as integrate

def f(x):
    return np.exp(-x**2)

# 1. Regla del Trapecio Compuesta
def trapecio_compuesto(func, a, b, n):
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = func(x)

    suma = y[0] + 2.0 * np.sum(y[1:-1]) + y[-1]
    return (h / 2.0) * suma

# 2. Regla de Simpson Compuesta
def simpson_compuesto(func, a, b, n):
    if n % 2 != 0:
        raise ValueError("El número de subintervalos n debe ser par para la regla de Simpson.")
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = func(x)

    suma = y[0] + 4.0 * np.sum(y[1:-1:2]) + 2.0 * np.sum(y[2:-2:2]) + y[-1]
    return (h / 3.0) * suma
```

```
# Parámetros
a, b = 0.0, 1.0
subintervalos = 20 # Debe ser par

# Integral de referencia de alta precisión usando scipy (método adaptativo QUAD)
valor_referencia, _ = integrate.quad(f, a, b)

res_trap = trapecio_compuesto(f, a, b, subintervalos)
res_simp = simpson_compuesto(f, a, b, subintervalos)

err_trap = abs(valor_referencia - res_trap)
err_simp = abs(valor_referencia - res_simp)

print(f"Valor de referencia exacto: {valor_referencia:.16f}\n")
print(f"Trapezio Compuesto (n={subintervalos}): {res_trap:.16f} | Error: {err_trap:.2e}")
print(f"Simpson Compuesto (n={subintervalos}): {res_simp:.16f} | Error: {err_simp:.2e}")
```

La regla de Simpson aprovecha la suavidad de la campana de Gauss para dar un error en el orden de 10^{-8} con solo 20 intervalos, mientras que la regla del Trapecio arroja un error mucho mayor de 10^{-4} con el mismo número de muestras.

5. Ejercicios Resueltos

Ejercicio 1

Aproximar el valor de la integral $\int_0^2 x^3 dx$ usando la Regla de Simpson Simple ($n = 2$). Comparar con el valor analítico real obtenido por la regla de Barrow.

Solución:

1. Cálculo analítico real (Barrow):

$$\int_0^2 x^3 dx = \left[\frac{x^4}{4} \right]_0^2 = \frac{2^4}{4} - \frac{0^4}{4} = \frac{16}{4} - 0 = 4$$

2. Aproximación por la Regla de Simpson Simple: El intervalo es $[a, b] = [0, 2]$. El punto medio es $m = \frac{a+b}{2} = \frac{0+2}{2} = 1$. Evaluamos la función $f(x) = x^3$ en los tres nodos:

- $f(a) = f(0) = 0^3 = 0$
- $f(m) = f(1) = 1^3 = 1$
- $f(b) = f(2) = 2^3 = 8$ Sustituyendo en la fórmula de Simpson Simple:

$$\int_0^2 x^3 dx \approx \frac{b-a}{6} [f(a) + 4f(m) + f(b)] = \frac{2-0}{6} [0 + 4(1) + 8] = \frac{2}{6} \cdot 12 = \frac{1}{3} \cdot 12 = 4$$

3. Comparación de resultados: El valor aproximado es exactamente 4, igual al valor real. Esto se debe a que la regla de Simpson tiene un error proporcional a la cuarta derivada de la función. Dado que la función $f(x) = x^3$ tiene derivadas de orden 4 nulas ($f^{(4)}(x) = 0$), el error de truncamiento de la regla de Simpson es exactamente cero.

6. Ejercicios Propuestos

1. Estimar el valor de $\int_1^3 \frac{1}{x} dx$ utilizando la Regla del Trapecio Compuesta con $n = 4$ subintervalos. Comparar el resultado aproximado con el valor exacto de la integral ($\ln 3 \approx 1.098612$).
2. Calcular una aproximación de $\int_0^\pi \sin(x) dx$ empleando la Regla de Simpson Compuesta con $n = 4$ intervalos y calcular el error absoluto cometido.
3. Determinar analíticamente el número mínimo de subintervalos n requeridos para aproximar la integral $\int_0^1 e^x dx$ mediante la Regla del Trapecio Compuesta con un error absoluto garantizado menor que 10^{-4} .

Glosario de Términos

- **Afijo:** Punto en el plano cartesiano (plano de Argand) que representa geométricamente a un número complejo $z = a + bi$ mediante las coordenadas (a, b) .
- **Argumento:** Ángulo en radianes formado por el vector representativo de un número complejo con el semieje real positivo.
- **Convergencia:** Propiedad matemática de una sucesión o serie que indica que sus términos se aproximan acumulativamente a un valor límite finito.
- **Diferencia Dividida:** Operador de diferencias recursivo empleado en el método de Newton para construir de forma eficiente el polinomio interpolador.
- **Discontinuidad Evitable:** Tipo de discontinuidad en la cual existe el límite de la función en el punto, pero no coincide con el valor de la función o esta no está definida en él.
- **Error de Redondeo:** Discrepancia aritmética generada al representar números reales infinitos bajo un número finito de bits en la memoria del computador (estándar IEEE 754).
- **Fenómeno de Runge:** Comportamiento oscilatorio inestable que se produce cerca de los extremos del intervalo de interpolación al ajustar un polinomio de grado alto sobre nodos equiespaciados.
- **Gradiente:** Vector de derivadas de una función. En optimización de una sola variable, el gradiente coincide con la derivada ordinaria y señala la dirección de máximo crecimiento local.
- **Integral de Riemann:** Formalización matemática del área bajo una curva construida mediante la suma de áreas de rectángulos elementales cuando el ancho de estos tiende a cero.
- **Módulo:** Longitud del vector representante de un número complejo, equivalente a la distancia euclídea del afijo (a, b) al origen de coordenadas: $r = \sqrt{a^2 + b^2}$.
- **Nodos de Chebyshev:** Distribución óptima no equiespaciada de puntos en un intervalo que reduce el error de aproximación polinómica y suprime el fenómeno de Runge.
- **Polinomio de Maclaurin:** Polinomio de Taylor centrado específicamente en el punto $a = 0$.
- **Polinomio de Taylor:** Desarrollo polinómico local de grado n que aproxima el valor de una función derivable en el entorno de un punto a .
- **Primitiva:** Función $F(x)$ cuya derivada corresponde a la función original dada: $F'(x) = f(x)$.
- **Resto de Lagrange:** Término matemático que cuantifica de forma exacta el error cometido al aproximar una función continua mediante un polinomio de Taylor truncado.
- **Suma de Kahan:** Algoritmo que compensa de forma dinámica la pérdida de bits significativos al acumular sumas repetidas de números en coma flotante simple o doble.

Bibliografía Recomendada

1. **Apostol, T. M. (1967).** *Calculus*, Volumen 1: Cálculo con funciones de una variable, con una introducción al álgebra lineal. Editorial Reverté.
 - *Nota:* Un clásico riguroso que detalla formalmente toda la teoría de límites, derivadas, integración e introducción a las series.
2. **Burden, R. L., & Faires, J. D. (2011).** *Análisis Numérico* (9ª ed.). Cengage Learning.
 - *Nota:* Texto de referencia para el estudio práctico de la bisección, Newton-Raphson, la interpolación de Lagrange/Newton y las reglas de Simpson/Trapezio.
3. **Larson, R., & Edwards, B. H. (2016).** *Cálculo* (10ª ed.), Volumen 1. Cengage Learning.
 - *Nota:* Muy didáctico, con gran cantidad de problemas visuales e interpretaciones geométricas de la continuidad y derivación.
4. **Spivak, M. (2008).** *Calculus* (4ª ed.). Publish or Perish.
 - *Nota:* Obra emblemática para comprender a fondo y formalmente las demostraciones detrás de los teoremas de Bolzano, Weierstrass, Rolle y del Valor Medio.
5. **Stewart, J. (2012).** *Cálculo de una variable: Trascendentes tempranas* (7ª ed.). Cengage Learning.
 - *Nota:* Excelente equilibrio entre rigor matemático y aplicaciones prácticas, muy recomendado para los bloques de límites e integración.